– LeMoS – Benutzerhandbuch

Aufbau dieses Dokuments

Grundlagen
- Kontext und allgemeine Informationen zum LeMoS

Vorgehensweise
- Arbeitsablauf zur Erstellung eines Lernmoduls

Erste Schritte
- Exemplarische Modellierung eines Lernmoduls

Modellierungssprache
- Spezifikation aller Konfigurationsmöglichkeiten des LeMoS

Systemfunktionen
- Spezifikation aller vom LeMoS bereitgestellten Funktionen

– LeMoS – Grundlagen

Kontext dieses Dokuments

- Zur Vereinfachung der Erstellung von Lernmodulen, sollen diese modelliert statt programmiert werden.
- Um dies zu ermöglichen wurde das Lernstoff-Modellierungs-System (LeMoS) geschaffen.
- Lernmodule werden modelliert, indem sie als Konfiguration beschrieben werden.
- Das LeMoS wandelt Konfigurationen in fertige webbasierte Lernmodule um.
- Konfigurationen werden in der Modellierungssprache des LeMoS beschrieben.

- Syntaktische Grundlage der Modellierungssprache ist die strukturierte Sprache YAML.
- Die Struktur der Modellierungssprache ist in dem Dokument "Modellierungssprache.pdf" definiert.
- Der Ablauf eines Lernmoduls wird als Zustandsgraph beschrieben.
- Die konfigurierbaren Aktionen, Operationen und Bedingungen sind in JavaScript-Syntax zu definieren.
- Zur Interaktion mit dem Lernmodul können die Systemfunktionen des LeMoS genutzt werden, die in "Systemfunktionen.pdf" definiert sind.

- Zum Erzeugen/Testen eines Lernmoduls ist die Anleitung "readme-lernmodulersteller.txt" zu befolgen.
- Alle benötigten Dateien (inklusive der genannten Dokumente) werden vom Entwickler des LeMoS bereitgestellt.

Überblick der Begrifflichkeiten

Benutzer

- Person, die die Lernmodule zum eigenständigen Lernen verwendet

Lernmodulersteller

- Person, die die Lerninhalte konzipiert und entsprechende Lernmodule modelliert

Lerninhalt

- Inhalt, der den Benutzern vermittelt werden soll

Lernmodul

- Webseite, die die Interaktion mit einem Lerninhalt ermöglicht

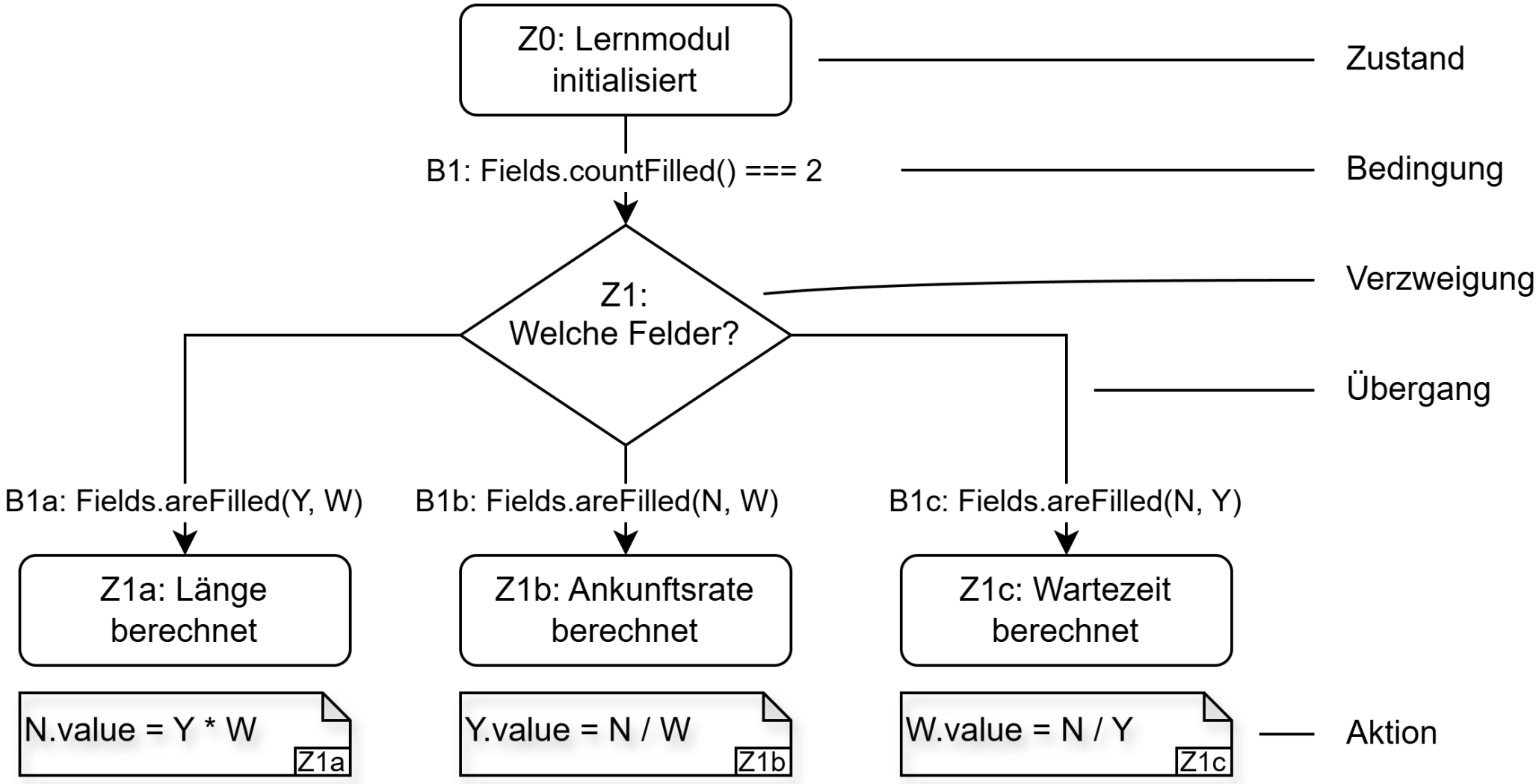
Lernplattform

- Webseite, auf der die Lernmodule angeboten werden

Konfiguration

- Beschreibung, aus der ein Lernmodul erzeugt werden soll

Aufbau der Zustandsgraphen



Funktionsweise der Zustandsgraphen

- Beschreiben den Ablauf eines Lernmoduls durch Verknüpfung mehrerer Zustände.
- Jeder Zustand kann Aktionen definieren, die vor seinem Eintritt ausgeführt werden.
- Jeder Zustand kann Übergänge zu anderen Zuständen definieren.
- Ausgangspunkt des Zustandsgraphen ist der genau einmal zu definierende Einstiegszustand.
- Endzustände werden nicht explizit definiert, der Zustandsgraph endet, sobald bei einem Zustand keine Übergänge möglich sind (entweder keine Übergänge definiert, oder keine der Bedingungen erfüllt).

- Verzweigungen können keine Aktionen definieren und werden stets automatisch durchgeführt.
- Verzweigungen funktionieren wie normale Zustände und sind daher auch stets durch normale Zustände ersetzbar, sie dienen ausschließlich der semantischen Abgrenzung.

- Zustandsübergänge können bedingt oder unbedingt sein.
- Pro Zustand darf maximal ein Zustand ohne Bedingung (unbedingt) sein.
- Es wird stets der erste Übergang durchgeführt, dessen Bedingung erfüllt ist.
- Unbedingte Übergänge werden stets als letztes betrachtet.

– LeMoS – Vorgehensweise

1. Konzeptionieren des Lernmoduls

Lernmodulersteller erstellt Konzept für Lernmodul:

- Was soll vom Lernmodul vermittelt werden?
- Welche Werte sollen im Lernmodul eingegeben und ausgegeben werden?
- Wie soll das Lernmodul zu dem gewünschten Ergebnis kommen (als Zustandsgraph)?

2. Modellieren des Lernmoduls

Lernmodulersteller überträgt Konzept in Modellierungssprache:

- Syntaktische Grundlage der Modellierungssprache: YAML
- Dokumentation der Struktur der Modellierungssprache in Kapitel "Modellierungssprache"
- Dokumentation der vom LeMoS bereitgestellten Funktionen in Kapitel "Systemfunktionen"
- Unterstützung durch Verwendung der Schemata möglich:
  - Schema "LeMoS-Modul-Schema.json" für die Konfiguration des Lernmodul
  - Schema "LeMoS-Szenario-Schema.json" für die Konfiguration der Lernmodul-Szenarien
  - Nähere Erläuterungen dazu in Kapitel "Erste Schritte"
  - Hinweis: das LeMoS validiert nicht gegen diese Schemata, daraus ergibt sich folgendes:
    - Sollte ein Fehler im Schema vorliegen, sodass etwas fälschlicherweise als inkorrekt klassifiziert wird, kann das LeMoS die Konfiguration ggf. dennoch verarbeiten
    - Das LeMoS kann auch Fehler erkennen, die das Schema nicht erkennt, etwa wenn es nicht genau einen ENTRY-State gibt, es ist daher ratsam auch die Ausgabe des LeMoS zu betrachten

3. Generieren des Lernmoduls

Lernmodulersteller nutzt LeMoS, um Konfiguration in fertiges Lernmodul umzuwandeln:

- Vorbedingungen:
  - Java JDK21 ist installiert und unter der Pfad-Variable "java" erreichbar
  - Die vom Softwareentwickler gelieferten Dateien liegen entpackt im Dateisystem vor
- Ausführen des zugehörigen Skripts an beliebiger Stelle:
  - Unter Linux: "run-lemos.sh"
  - Unter Windows: "run-lemos.bat"
- Parameter beim Aufruf des Skripts: absoluter oder relativer Pfad der Lernmodulkonfiguration (Angabe mehrerer Lernmodulkonfigurationen möglich)
- Ergebnis des Skripts: ZIP-Archiv mit dem Namen der Lernmodulkonfiguration, abgelegt am Ausführungsort des Skripts

3. Generieren des Lernmoduls – Beispiel

Ordnerstruktur:

- Überordner
  - LeMoS
    - lemos-<version>.jar (z.B. lemos-1.0.0.jar) // Vom Softwareentwickler des LeMoS bereitzustellen
    - run-lemos.sh // Vom Softwareentwickler des LeMoS bereitzustellen
    - run-lemos.bat // Vom Softwareentwickler des LeMoS bereitzustellen
  - Konfigurationen
    - Benutzerhandbuch.pdf // Vom Softwareentwickler des LeMoS bereitzustellen
    - LeMoS-Modul-Schema.json // Vom Softwareentwickler des LeMoS bereitzustellen
    - LeMoS-Szenario-Schema.json // Vom Softwareentwickler des LeMoS bereitzustellen
    - Modul_Subtraktion
      - Subtraktion.yaml
      - scenarios (Optional)
        - Subtraktion-S1.yaml
        - ...
      - resources (Optional)
        - ...
    - ...
  - Zielordner

- Aufruf des Skripts im Zielordner: *../LeMoS/run-lemos.sh ../Konfigurationen/Modul_Subtraktion/Subtraktion.yaml*
- Ergebnis im Zielordner: *Subtraktion.zip*

4. Bereitstellen des Lernmoduls

Lernmodulersteller testet erzeugtes Lernmodul und lässt es bereitstellen:

- Vorbedingungen:
  - Der Browser muss zum Ausführen des Lernmoduls mindestens die ECMAScript-Version es2022 unterstützen
  - Falls der Browser 'Mozilla Firefox' verwendet wird, darf dieser nicht via Snap installiert sein, da hier der Funktionsumfang ungenügend ist
- Testen des Lernmoduls in einer der folgenden Varianten (direktes öffnen im Browser ist nicht möglich):
  - Via Apache:
    - Verschieben der entpackten ZIP-Datei an den gewünschten Document-Root
    - Apache entsprechend des Document-Root konfigurieren
  - Via Tomcat:
    - Verschieben der entpackten ZIP-Datei in den webapps-Ordner von Tomcat
    - Alternativ: Umbenennen der ZIP-Datei zu einer WAR-Datei und verschieben in den webapps-Ordner von Tomcat
    - URL des Lernmoduls: <Webserver-URL>/<Lernmodul-Dateiname> --> Beispiel:
      - URL des Webservers: *http://localhost:8080*
      - Name des entpackten Ordners bzw. der Verknüpfung im webapps-Ordner: *Subtraktion*
      - URL des Lernmoduls: *http://localhost:8080/Subtraktion*
  - Via Tomcat GUI:
    - Öffnen der Seite http://localhost:8080/manager in einem Webbrowser
    - Unter "Lokale WAR Datei zur Installation hochladen" auf "Durchsuchen" klicken und ZIP-Datei auswählen
    - Unter "Lokale WAR Datei zur Installation hochladen" auf "Installieren" klicken
    - Unter "Anwendungen" auf Kontext-Pfad (entspricht /<Lernmodul-Dateiname>) des Lernmoduls klicken --> Beispiel:
      - Name der ZIP-Datei: *Subtraktion.zip*
      - Kontext-Pfad des Lernmoduls: */Subtraktion*
- Bereitstellung: Weiterleiten des generierten ZIP-Archivs an den Verwalter der Lernplattform

– LeMoS –
Erste Schritte

Erste Schritte

Vorüberlegungen

Lerninhalt

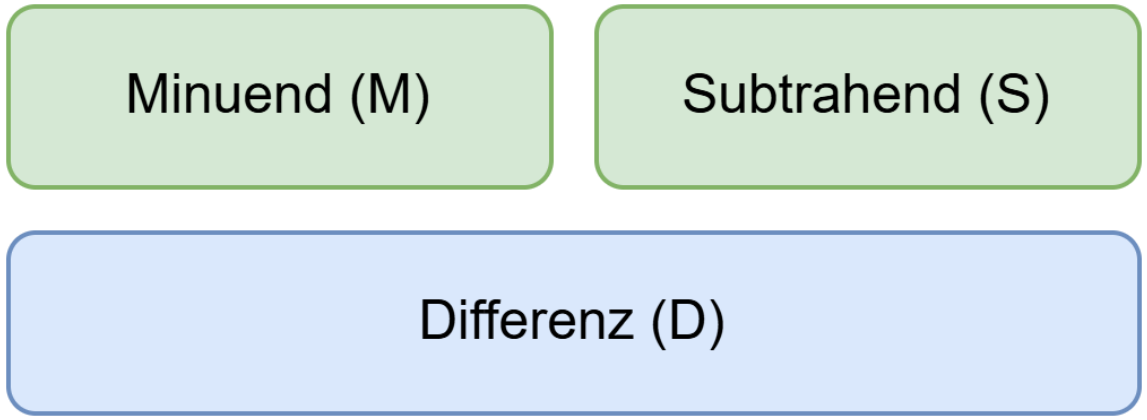
Was soll das neue Lernmodul vermitteln?

- Konzept der Subtraktion (Minuend minus Subtrahend ergibt die Differenz)
- Gegeben (vom Benutzer einzugeben) sind somit folgende Werte:
  - Minuend (M), beispielsweise 7
  - Subtrahend (S), beispielsweise 5
- Gesucht (vom Lernmodul zu berechnen) ist somit folgender Wert:
  - Differenz (D), im Beispiel: 2
- Mögliche Abläufe des Lernmoduls
  - Variante 1: Berechnung in einem Schritt ohne weitere Bedingungen:
    1. Abziehen des Subtrahenden vom Minuenden, um Differenz zu erhalten
  - Variante 2: Minuend muss größer als der Subtrahend sein:
    1. Wenn Minuend größer als Subtrahend: Differenz = Minuend – Subtrahend
    2. Ansonsten: mache nichts
  - Variante 3: Es wird stets der kleinere vom größeren Wert abgezogen (Betrag der Differenz):
    1. Wenn Minuend größer als Subtrahend: Differenz = Minuend – Subtrahend
    2. Ansonsten: Differenz = Subtrahend - Minuend

Bedienoberfläche

Welche Felder werden zur Interaktion mit dem Lernmodul benötigt?

- Benutzer muss die beiden Ausgangswerte (Minuend und Subtrahend) eingeben können, daher wird für beide ein Eingabefeld benötigt.
- Benutzer soll das Ergebnis des Lernmoduls (Differenz) in weiterem Feld erhalten.
- Anordnung: die beiden Eingabefelder sollen nebeneinander, das Ausgabefeld darunter positioniert sein.

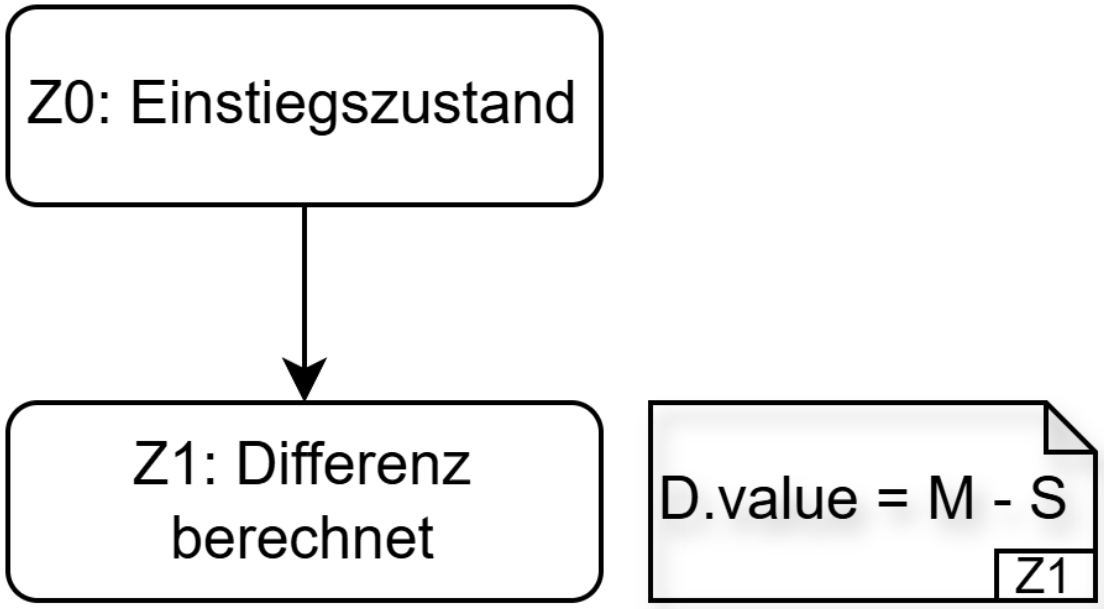


Schematische Darstellung der Bedienoberfläche (grün: Eingabefelder; blau: Ausgabefeld)

Zustandsgraph (Variante 1)

Wie kann der Ablauf des Lernmodul als Zustandsgraph beschrieben werden?

- Der Ablauf der ersten Variante sieht nur einen Schritt (Berechnung der Differenz) vor.
- Neben dem initialen Zustand des Lernmoduls (Einstiegszustand Z0) gibt es somit nur noch den zur Berechnung (Z1)
- Letzterer Zustands soll dafür die Operation Differenz (D) = Minuend (M) - Subtrahend (S) durchführen.
- Um den berechneten Wert dem Differenzfeld D zuzuweisen ist im LeMoS die Ergänzung ".value" zu verwenden.
- Die in Z1 auszuführende Aktion lautet somit D.value = M - S.

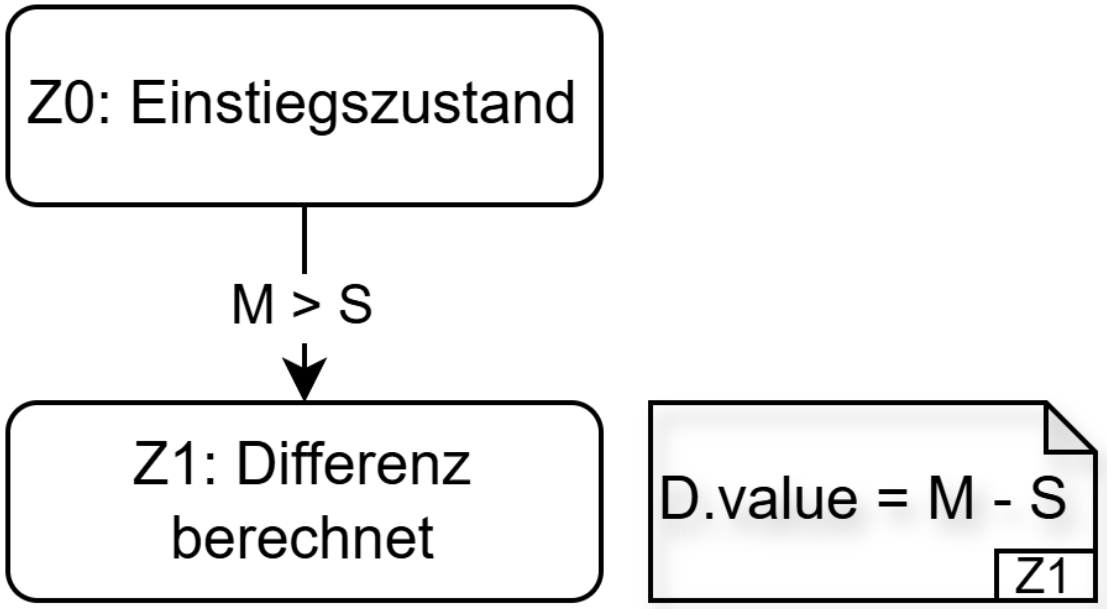


Darstellung des Zustandsgraphen für Variante 1

Zustandsgraph (Variante 2)

Wie kann der Ablauf des Lernmodul als Zustandsgraph beschrieben werden?

- Ergänzend zur ersten Variante soll die Berechnung in der zweiten Variante nur noch erfolgen, wenn der Minuend M größer als der Subtrahend S ist.
- Der bisher bedingungslose Übergang von Z0 (Einstiegszustand) zu Z1 (Berechnung) wird daher um die Bedingung M > S erweitert.

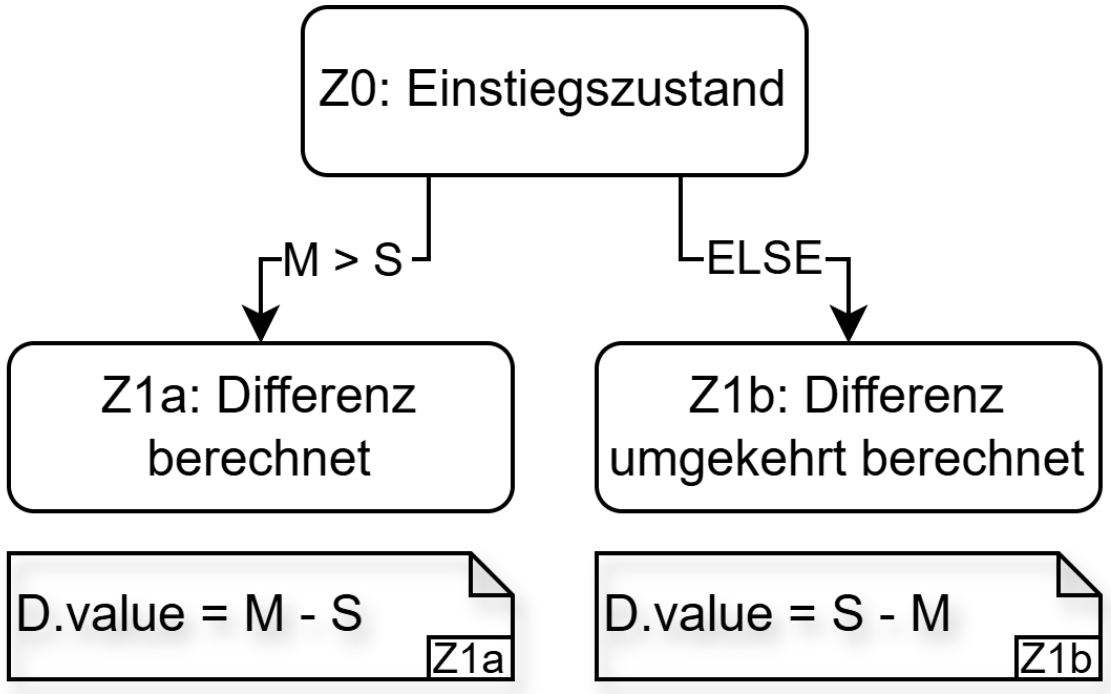


Darstellung des Zustandsgraphen für Variante 2

Zustandsgraph (Variante 3)

Wie kann der Ablauf des Lernmodul als Zustandsgraph beschrieben werden?

- Ergänzend zur zweiten Variante soll das Lernmodul in der dritten Variante nicht enden, wenn der Minuend M nicht größer als der Subtrahend S ist.
- Stattdessen soll die Differenz in diesem Fall umgekehrt, das heißt Subtrahend minus Minuend berechnet werden.
- Der Zustandsgraph wird daher um einen weiteren Zustand (Z1b) sowie einen von Z0 zu ihm führenden Übergang ergänzt.
- Der bisherige Zustand Z1 wird zur Klarstellung in Z1a umbenannt.
- Der Übergang zu Z1b soll genau dann erfolgen, wenn der zu Z1a aufgrund der Bedingung nicht erfolgen konnte, weitere Bedingungen sollen nicht aufgestellt werden.



Darstellung des Zustandsgraphen für Variante 3

Erste Schritte

Lernmodulkonfiguration

1. Grundstruktur mit Schema definieren

Grundstruktur für jedes Lernmodul:

```yaml
# yaml-language-server: $schema=
general:
fields:
states:
```

Ergänzen des Pfads zum Schema für Lernmodulkonfigurationen des LeMoS:
(Beispiel: Schema erreichbar mittels ../LeMoS-Modul-Schema.json)

```yaml
# yaml-language-server: $schema=../LeMoS-Modul-Schema.json
general:
fields:
states:
```

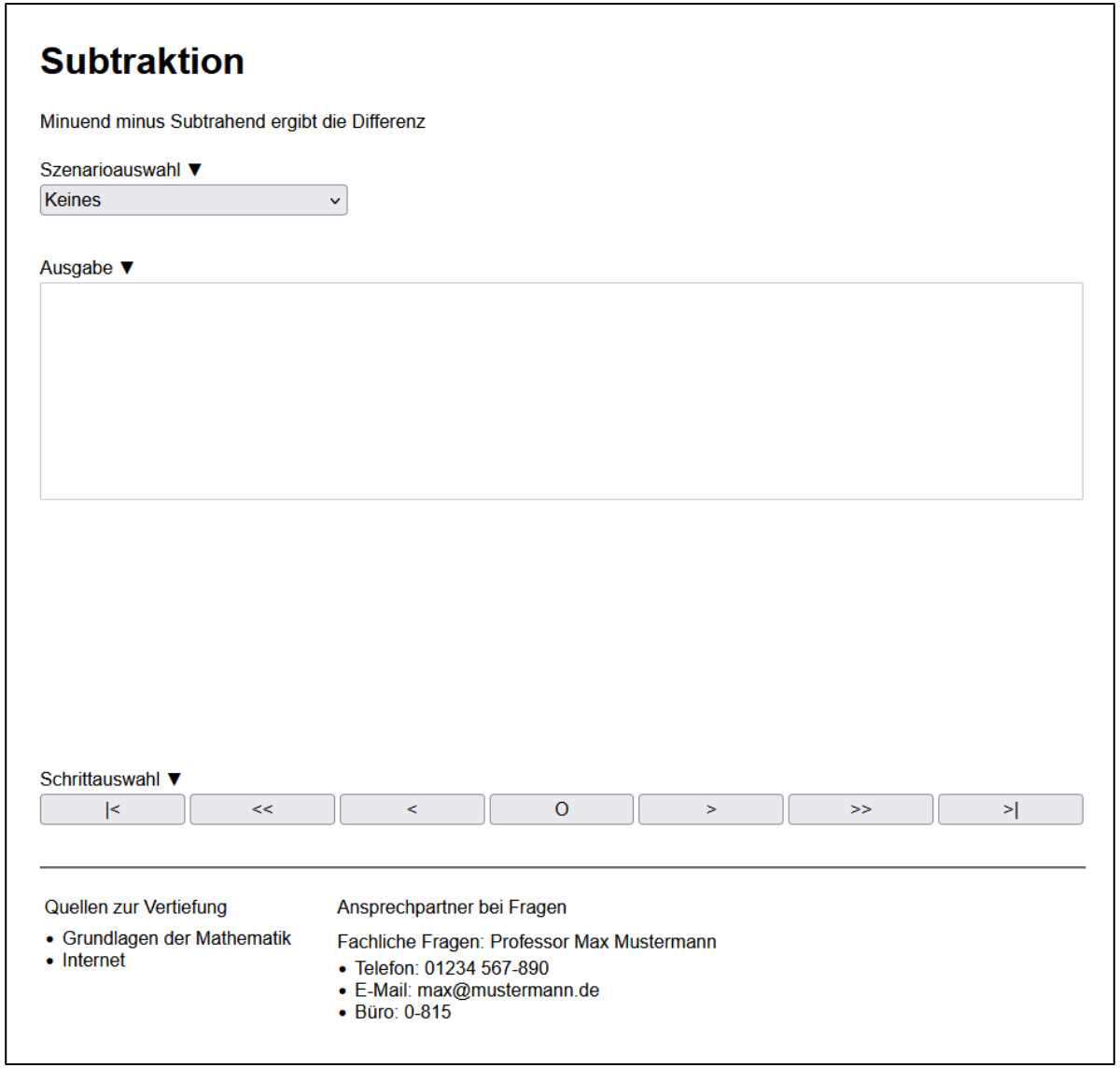
2. Allgemeine Angaben definieren

- Erforderliche Attribute:
  - heading für den Namen / die Überschrift des Lernmoduls (Bsp.: Subtraktion)
- Empfohlene Attribute:
  - explanation für Erläuterungen zum Lernmodul (Bsp.: Minuend minus Subtrahend ergibt die Differenz)
  - sources für Quellenangaben (Bsp.: Skript zu Grundlagen der Mathematik oder Internet)
  - contacts für Kontaktangaben (Bsp.: Beliebige Angaben für fachliche Fragen an Herrn Mustermann)
  - settings/lemosVersion zur Angabe der verwendeten Version des LeMoS für spätere Aufwärtskompatibilität (Bsp.: Version 2.0.0)
- Weitere Möglichkeiten:
  - Weitere Einstellungen des Moduls über settings
  - Einstellungen des Protokollierungsfelds über logger
  - Definieren globaler Variablen über variables
  - Definieren eigener Funktionen über functions
  - Importieren zusätzlicher Bibliotheken über imports
  - Hinterlegen zusätzlicher Ressourcen wie Bilder/Audios/Videos über resources

2. Allgemeine Angaben definieren – Beispielkonfiguration

```yaml
general:
  heading: "Subtraktion"
  explanation: "Minuend minus Subtrahend ergibt die Differenz"
  sources:
    - "Grundlagen der Mathematik"
    - "Internet"
  contacts:
    - name: "Professor Max Mustermann"
      purpose: "Fachliche Fragen"
      data:
        Telefon: "01234 567-890"
        E-Mail: "max@mustermann.de"
        Büro: "0-815"
  settings:
    lemosVersion: "2.0.0"
```

2. Allgemeine Angaben definieren – Ergebnis

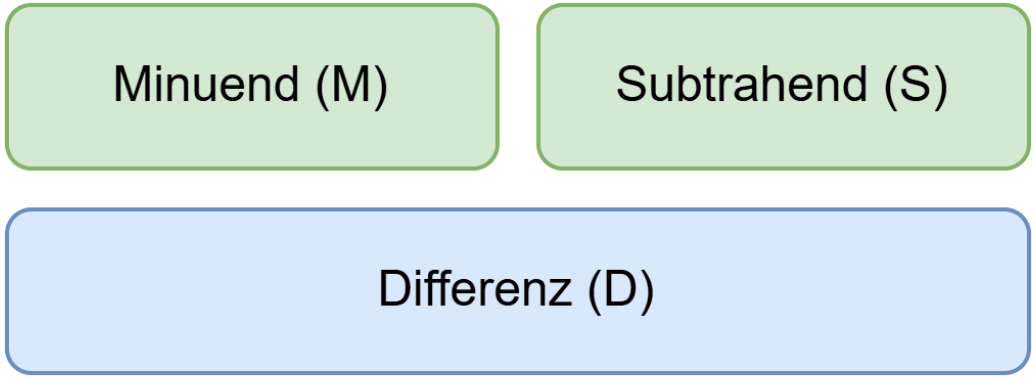


3. Felder definieren

- Erforderliche Attribute (pro Feld):
  - id: eindeutige Bezeichnung des Felds, unter der es im Lernmodul angesprochen werden soll (Bsp.: der Minuend soll M, der Subtrahend S und die Differenz D heißen)
  - type: Art des Feldes wie Infofeld, Textfeld, Listenfeld, …
    (Bsp.: alle drei Felder, M, S und D sollen einfache Textfelder, d.h. type TEXT sein)
- Empfohlene Attribute (pro Feld):
  - name: Anzeigename des Felds (Bsp.: Minuend, Subtrahend oder Differenz)
  - explanation: Erläuterung zum Feld (Bsp.: linker/rechter Ausgangswert zur Bildung der Differenz)
  - usage: Verwendungsart des Felds (Bsp.: Differenzfeld benötigt keine Eingabe, daher usage OUTPUT)
  - style: Größe und Positionierung des Felds (Bsp.: Minuend und Subtrahend nebeneinander, Differenz darunter)
  - Für Eingabefelder (usage INPUT):
    - datatype: Datentyp des Felds (Bsp.: es sind nur Zahlen erlaubt, daher datatype NUMBER)
    - restrictions: Weitere Einschränkungen der Eingaben (Bsp.: es sind nur positive Werte erlaubt)
- Weitere Möglichkeiten (pro Feld):
  - Ausblenden des Felds über hidden (standardmäßig sichtbar)
  - Hervorheben des Felds über highlighted (standardmäßig nicht hervorgehoben)
  - Für Eingabefelder: Einstellen, ob das Feld leer bleiben darf über allowEmpty (standardmäßig nicht erlaubt)
  - Für Eingabefelder: Konfigurieren von Reaktionen auf Eingaben über reactions (standardmäßig keine)
  - Weitere Attribute und Konfigurationsmöglichkeiten in Abhängigkeit von den Feldtypen (siehe type-Attribut)

3. Felder definieren – Style-Attribut

- Erforderliche Attribute:
  - row: Vertikale Positionierung des Felds (Zeile im Raster, beginnend bei 1)
    (Bsp.: M und S sollen in erster Zeile, D darunter in der zweiten Zeile positioniert werden)
  - column: Horizontale Positionierung des Felds (Spalte im Raster, beginnend bei 1)
    (Bsp.: M und D sollen am Anfang ihrer Zeile stehen, S soll erst in der zweiten Spalte folgen)
- Optionale Attribute:
  - width: Horizontale Ausdehnung des Felds (Anzahl der belegten Spalten)
    (Bsp.: D soll so breit sein, wie M und S zusammen und muss daher zwei Spalten breit sein)
  - height: Vertikale Ausdehnung des Felds (Anzahl der belegten Zeilen)
    (Bsp.: alle Felder sollen gleichhoch sein, der Wert muss daher nicht konfiguriert werden)
- Hinweis: Style-Attribut muss entweder für alle oder für keines der Felder definiert werden



Im Beispiel gewünschte Darstellung der Felder

3. Felder definieren – Beispielkonfiguration (Minuend)

```yaml
fields:
  - id: "M"
    name: "Minuend"
    explanation: "Linker Ausgangswert zur Bildung der Differenz"
    type: TEXT
    datatype: NUMBER
    restrictions:
      - errorMessage: "Nur positive Zahlenwerte, ${M.value} ist negativ"
        terms:
          - "M >= 0"
    style:
      row: 1
      column: 1
```

3. Felder definieren – Beispielkonfiguration (Subtrahend)

```yaml
fields:
  - id: "S"
    name: "Subtrahend"
    explanation: "Rechter Ausgangswert zur Bildung der Differenz"
    type: TEXT
    datatype: NUMBER
    restrictions:
      - errorMessage: "Nur positive Zahlenwerte, ${S.value} ist negativ"
        terms:
          - "S >= 0"
    style:
      row: 1
      column: 2
```

3. Felder definieren – Beispielkonfiguration (Differenz)

```yaml
fields:
  - id: "D"
    name: "Differenz"
    explanation: "Resultat der Subtraktion der beiden Ausgangswerte"
    type: TEXT
    usage: OUTPUT
    style:
      row: 2
      column: 1
      width: 2
```

3. Felder definieren – Beispielkonfiguration (Gesamt)

```yaml
fields:
  - id: "M"
    name: "Minuend"
    explanation: "Linker Ausgangswert zur Bildung der Differenz"
    # Weitere konfigurierte Eigenschaften …
  - id: "S"
    name: "Subtrahend"
    explanation: "Rechter Ausgangswert zur Bildung der Differenz"
    # Weitere konfigurierte Eigenschaften …
  - id: "D"
    name: "Differenz"
    explanation: "Resultat der Addition der beiden Ausgangswerte"
    # Weitere konfigurierte Eigenschaften …
```

3. Felder definieren – Ergebnis

Subtraktion

Minuend minus Subtrahend ergibt die Differenz

Szenarioauswahl ▼

Keines

Minuend ▲

Linker Ausgangswert zur Bildung der Differenz

Subtrahend ▲

Rechter Ausgangswert zur Bildung der Differenz

Differenz ▲

Resultat der Subtraktion der beiden Ausgangswerte

Ausgabe ▼

Schrittauswahl ▼

|< << < O > >> >|

Quellen zur Vertiefung

- Grundlagen der Mathematik
- Internet

Ansprechpartner bei Fragen

Fachliche Fragen: Professor Max Mustermann

- Telefon: 01234 567-890
- E-Mail: max@mustermann.de
- Büro: 0-815

4. Zustände definieren

- Erforderliche Attribute (pro Zustand):
  - id: eindeutige Bezeichnung des Zustands, unter dem er im Lernmodul referenziert werden soll (Bsp.: der Einstiegszustand heißt Z0, der Zustand zur Berechnung der Differenz Z1)
- Empfohlene Attribute (pro Zustand):
  - type: Art des Zustands, ENTRY für Einstiegszustand (muss genau einmal pro Lernmodul vorkommen), JUNCTION für reine Fallunterscheidungen (dient zur semantischen Unterscheidung zu normalen Zuständen), STATE (Standardwert) für allgemeine Zustände (Bsp.: für Z0: type ENTRY, für Z1 type STATE)
  - importance: Wichtigkeit des Zustands, d.h. bei welchen Schrittweiten soll nach dem Zustand automatisch fortgesetzt werden, ZERO, LOW oder standardmäßig HIGH (Bsp.: nicht definiert und somit HIGH)
  - actions: Bei Eintritt des Zustands auszuführende Aktionen (Bsp.: Berechnung der Summe in Z1)
  - transitions: Übergänge zu weiteren Zuständen (Bsp.: Übergang in Z0 zu Z1)
- Weitere Attribute (pro Zustand):
  - name: Zustandsname für interne Ausgaben (entspricht standardmäßig der ID)
  - explanation: Erläuterungen zum Zustand für den Benutzer
    (Wenn konfiguriert: Ausgabe über Protokollfeld, ansonsten keine Ausgabe)
  - errorMessage: Fehlermeldung, falls kein Übergang möglich ist
    (Wenn nicht konfiguriert wird keine Fehlermeldung ausgelöst)

4. Zustände definieren – Transitions-Attribut

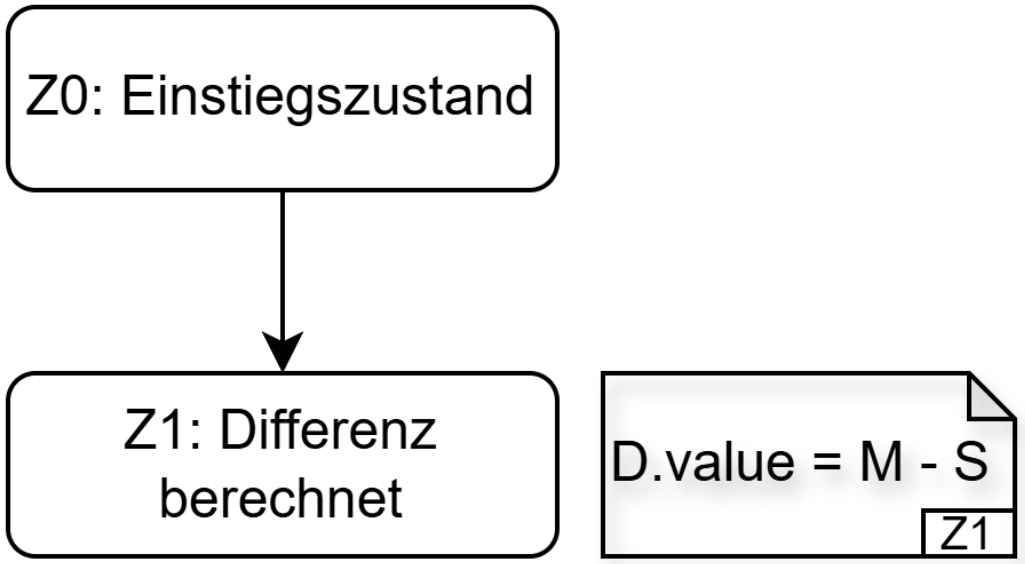
- Erforderliche Attribute (pro Zustand):
  - target: eindeutige Bezeichnung (id-Attribut) des Zielzustands des Übergangs
    (Bsp.: der Einstiegszustand Z0 hat in seinem einzigen Übergang den Zielzustand Z1)
- Empfohlene Attribute (pro Zustand):
  - conditions: Bedingungen, die für den Übergang zu erfüllen sind
    (Bsp. (Variante 2): Übergang von Z1 nur, wenn Minuend (M) > Subtrahend (S), ansonsten kein Übergang)
- Weitere Attribute (pro Zustand):
  - name: Name des Übergangs für interne Ausgaben (standardmäßig leer)
  - explanation: Erläuterungen zum Übergang für den Benutzer
    (Wenn konfiguriert: Ausgabe über Protokollfeld, ansonsten keine Ausgabe)

4. Zustände definieren – Conditions-Attribut

- Erforderliche Attribute (pro Zustand):
  - terms: Teilbedingungen, die erfüllt sein müssen
    (Bsp. (Variante 2): Minuend (M) muss größer als Subtrahend (S) sein, daher: M > S)
- Weitere Attribute (pro Zustand):
  - errorMessage: Fehlermeldung, falls die Bedingung nicht erfüllt ist
    (Wenn nicht konfiguriert: keine Fehlermeldung)

4. Zustände definieren – Beispielkonfiguration (Variante 1)

```yaml
states:
  - id: "Z0"
    name: "Einstiegszustand"
    type: ENTRY
    transitions:
      - target: "Z1"
  - id: "Z1"
    name: "Differenz berechnet"
    actions:
      - "D.value = M - S"
```

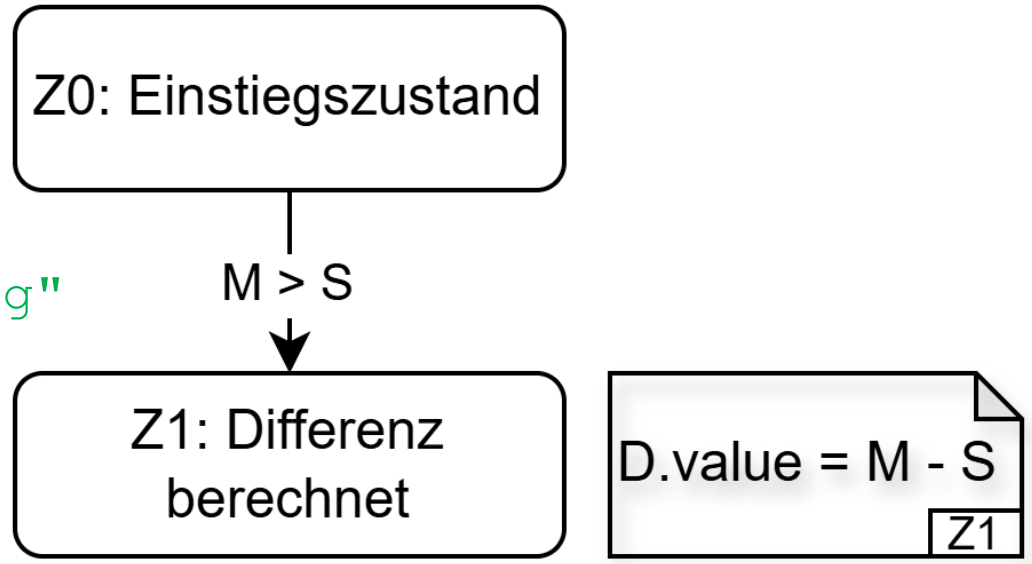


Zustandsgraph in Variante 1

4. Zustände definieren – Beispielkonfiguration (Variante 2)

Erinnerung: Übergang zu Z1 nur, wenn Minuend (M) > Subtrahend (S), ansonsten kein Übergang

```yaml
states:
  - id: "Z0"
    name: "Einstiegszustand"
    type: ENTRY
    transitions:
      - target: "Z1"
        conditions:
          - errorMessage: "Minuend zu gering"
            terms:
              - "M > S"
  - id: "Z1"
    name: "Differenz berechnet"
    actions:
      - "D.value = M - S"
```

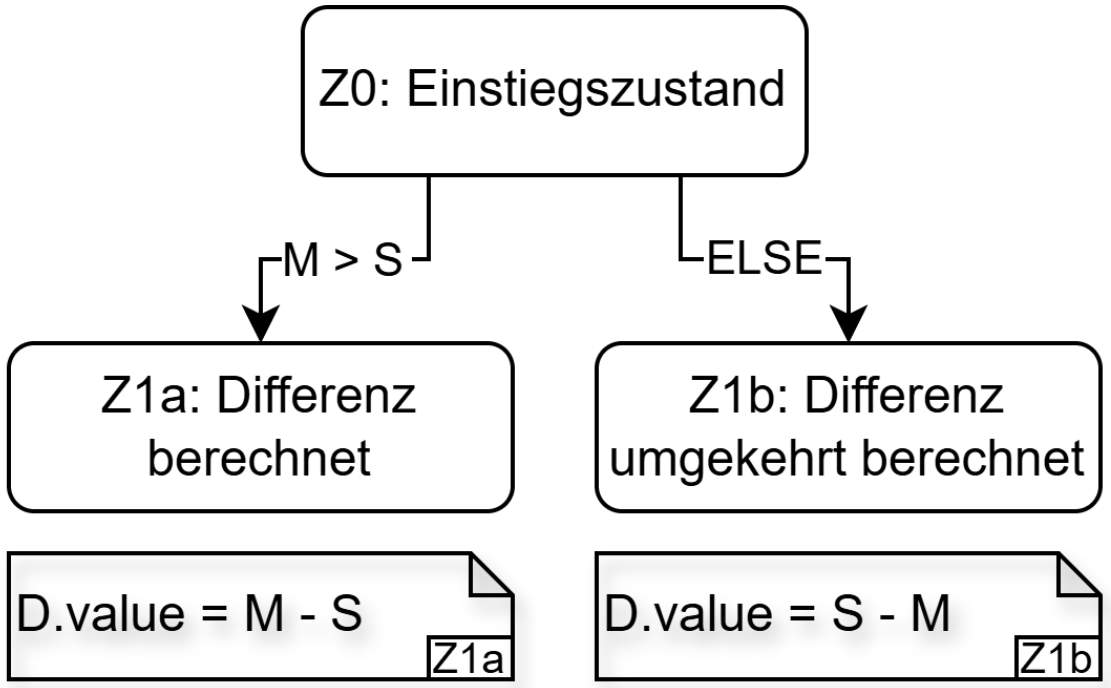


Zustandsgraph in Variante 2

4. Zustände definieren – Beispielkonfiguration (Variante 3)

Erinnerung: Übergang zu Z1a, wenn Minuend (M) > Subtrahend (S), ansonsten Übergang zu Z1b

```yaml
states:
  - id: "Z0"
    name: "Einstiegszustand"
    type: ENTRY
    transitions:
      - target: "Z1a"
        conditions:
          - terms:
              - "M > S"
      - target: "Z1b"
  - id: "Z1a"
    actions:
      - "D.value = M - S"
  - id: "Z1b"
    actions:
      - "D.value = S - M"
```



Zustandsgraph in Variante 3

5. Lernmodulkonfiguration zusammensetzen

```yaml
# yaml-language-server: $schema=../LeMoS-Modul-Schema.json
general:
  heading: "Subtraktion"
  explanation: "Minuend minus Subtrahend ergibt die Differenz"
  # Weitere konfigurierte Eigenschaften …
fields:
  - id: "M"
    name: "Minuend"
    # Weitere konfigurierte Eigenschaften …
  - id: "S"
    name: "Subtrahend"
    # Weitere konfigurierte Eigenschaften …
  - id: "D"
    name: "Differenz"
´   # Weitere konfigurierte Eigenschaften …
states:
  - id: "Z0"
    name: "Einstiegszustand"
    # Weitere konfigurierte Eigenschaften …
  - id: "Z1"
    name: "Differenz berechnet"
    # Weitere konfigurierte Eigenschaften …
```

Erste Schritte

Szenariokonfiguration

1. Unterordner anlegen

- Szenarien müssen im Unterordner "scenarios" abgelegt werden
- Beispiel:
  - Pfad der Lernmodulkonfiguration: /Lernmodule/Subtraktion/Subtraktion.yaml
  - Pfad des Unterordners: /Lernmodule/Subtraktion/scenarios
  - Pfad eines Szenarios: /Lernmodule/Subtraktion/scenarios/Subtraktion-S1.yaml
- Hinweis: jedes Szenario wird in einer eigenen Datei beschrieben

2. Grundstruktur mit Schema definieren

Grundstruktur für jedes Szenario:

```
# yaml-language-server: $schema=
scenario:
```

Ergänzen des Pfads zum Schema für Szenariokonfigurationen des LeMoS:
(Beispiel: Schema erreichbar mittels ../../LeMoS-Szenario-Schema.json)

```
# yaml-language-server: $schema=../../LeMoS-Szenario-Schema.json
scenario:
```

3. Szenario definieren

- Erforderliche Attribute:
  - id: eindeutige Bezeichnung des Szenarios, unter dem es im Lernmodul referenziert werden soll (Bsp.: S1)
  - initialization: Operationen die zum Initialisieren des Szenarios durchzuführen sind (Bsp.: Minuend (M) auf den Wert 7 und Subtrahend (S) auf den Wert 5 setzen, die Differenz (D) wird vom Lernmodul berechnet und daher nicht im Voraus definiert, das Differenz-Feld bleibt somit leer)
- Empfohlene Attribute:
  - name: Anzeigename des Szenarios in der Szenarioauswahl (Bsp.: Szenario 1)
  - explanation: Erklärungen zum Szenario (Bsp.: Sieben minus Fünf)

```yaml
scenario:
  id: "S1"
  name: "Szenario 1"
  explanation: "Sieben minus Fünf"
  initialization:
    - "M.value = 7"
    - "S.value = 5"
```

4. Szenariokonfiguration zusammensetzen

```yaml
# yaml-language-server: $schema=../../LeMoS-Szenario-Schema.json
scenario:
  id: "S1"
  name: "Szenario 1"
  explanation: "Sieben minus Fünf"
  initialization:
    - "M.value = 7"
    - "S.value = 5"
```

5. Gesamtergebnis betrachten

Subtraktion

Minuend minus Subtrahend ergibt die Differenz

Szenarioauswahl ▼

Szenario 1 (Sieben minus Fünf)

Minuend ▼

7

Subtrahend ▼

5

Differenz ▼

Ausgabe ▼

Schrittauswahl ▼

|< << < O > >> >|

Quellen zur Vertiefung

- Grundlagen der Mathematik
- Internet

Ansprechpartner bei Fragen

Fachliche Fragen: Professor Max Mustermann

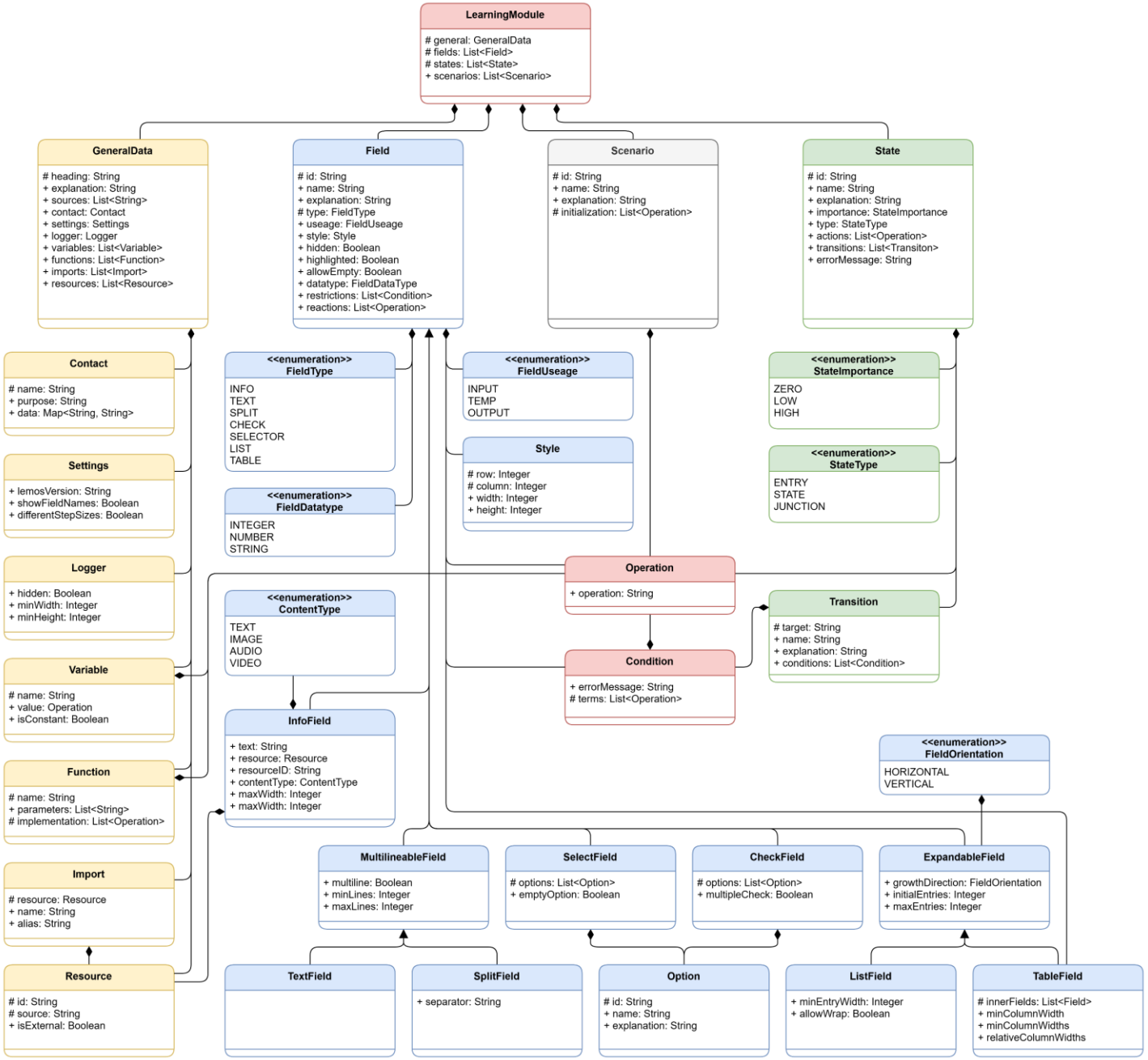
- Telefon: 01234 567-890
- E-Mail: max@mustermann.de
- Büro: 0-815

– LeMoS – Modellierungssprache

Grundlegende Struktur

Legende:

- '#' = Erforderliches Attribut
- '+‘ = Optionales Attribut



Modellierungssprache

Mehrfach genutzte Objekte

LearningModule

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
general	GeneralData	Allgemeine Angaben	Ja	Keiner
fields	List<Field>	Ein-/Ausgabefelder	Ja	Keiner
states	List<State>	Zustandsgraph	Ja	Keiner
scenarios	List<Scenario>	Beispielwerte	Nein	Leere Liste

Condition

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
errorMessage*	String	Fehlermeldung, falls Bedingung nicht erfüllt	Nein	Leere Zeichenkette (keine Fehlermeldung)
terms	List<Operation>	Zu erfüllende Teilbedingungen	Ja	Keiner

*: um Anweisungen in die Nachricht aufzunehmen, kann die ${}-Syntax von JavaScript verwendet werden,
Beispiel einer Konfigurationszeile: *errorMessage: Der Inhalt des Felds lautet ${feld.getValue()}, sollte aber … lauten.*

Operation

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
Keiner	String	Einzelner Befehl in JavaScript-Syntax (kann Felder, System- und Spezialfunktionen nutzen)	Ja	Keiner

Hinweis: als Fluchtzeichen müssen statt einem einzelnen Rück-Schrägstrich (\) zwei Rück-Schrägstriche verwendet werden (\\)

Modellierungssprache

Objekte für allgemeine Angaben

GeneralData

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
heading	String	Überschrift des Lernmoduls	Ja	Keiner
explanation	String	Erläuterungen zum Lerninhalt	Nein	Leere Zeichenkette
sources	List<String>	Quellen zur weiteren Recherche	Nein	Leere Liste
contacts	List<Contact>	Ansprechpartner für Fragen	Nein	Leere Liste
settings	Settings	Einstellungen des Lernmoduls	Nein	Siehe Settings
logger	Logger	Einstellungen der Protokollierung	Nein	Siehe Logger
variables	List<Variable>	Globale Variablen	Nein	Leere Liste
functions	List<Function>	Spezialfunktionen	Nein	Leere Liste
imports	List<Import>	Zusätzliche Bibliotheken	Nein	Leere Liste
resources	List<Resource>	Ressourcen für Infofelder	Nein	Leere Liste

Contact

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
name	String	Name des Kontakts	Ja	Keiner
purpose	String	Zuständigkeit des Kontakts (für welche Fragen)	Nein	Leere Zeichenkette
data	Map<String, String>	Beliebige Kontaktangaben (Ansprechpartner, E-Mail-Adresse, …)	Nein	Leere Tabelle

Settings

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
lemosVersion	String	Erwartete Version des LeMoS	Nein	Leere Zeichenkette (aktuelle Version)
showFieldIDs	Boolean	Zusätzliche Beschriftung der Felder mit ihrer ID	Nein	False
differentStepSizes	Boolean	Verschiedene Schrittweiten freischalten	Nein	True

Logger

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
hidden	Boolean	Protokollierungsfeld ausblenden	Nein	False
minWidth	Integer	Minimale Zeichenanzahl pro Zeile	Nein	100
minHeight	Integer	Minimale Zeilenanzahl	Nein	10

Variable

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
name	String	Eindeutiger Name	Ja	Keiner
value*	Operation	Initialer Wert	Nein	Leer (undefined)
isConstant	Boolean	Variable dient als Konstante	Nein	False

*: Muss konfiguriert werden wenn die Variable als Konstante dient (isConstant = true)

Function

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
name	String	Eindeutiger Name	Ja	Keiner
parameters	List<String>	Parameternamen	Nein	Leere Liste
implementation	List<Operation>	Implementierung	Ja	Keiner

Import

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
resource	Resource	ID der Ressource, die importiert werden soll	Ja	Keiner
name*	String	Zu importierende Komponente	Nein	Leere Zeichenkette (Standard-Import)
alias**	String	Bezeichnung unter der die Komponente angesprochen wird	Nein	Leere Zeichenkette

*: Standard-Import muss von importierter Ressource via Standard-Export unterstützt werden
**: Muss konfiguriert werden, wenn 'name‘ den Standard-Import referenziert

Hinweis: die statischen Systemfunktionen der Klassen Module, Fields, Resource und Utils müssen bei Bedarf importiert werden:

- Für Module: import Module from "../lib/systemFunctions/Module.js";
- Für Fields: import Fields from "../lib/systemFunctions/Fields.js";
- Für Resource: import Resource from "../lib/systemFunctions/Resource.js";
- Für Utils: import Utils from "../lib/systemFunctions/Utils.js";

Resource

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
id	String	Eindeutige Identifikation	Ja	Keiner
source	String	URL oder Pfad zur Ressource	Ja	Keiner
isExternal	Boolean	Ressource ist nicht im Lernmodul abgelegt	Nein	False

Hinweise:

- Lokale Ressourcen sind als Dateien im Unterordner „resources" des Lernmoduls abzulegen
- Der Pfad zu einer lokalen Ressource beginnt im resources-Ordner, Beispiel:
  - Ressource in Lernmodulordner/resources/example/example.js
  - Anzugebender Pfad: example/example.js

Modellierungssprache

Objekte für Felder

Field (1/2)

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
id	String	Eindeutige Identifikation, z.B. 'N'	Ja	Keiner
name	String	Anzeigename, z.B. 'Länge'	Nein	Wert von 'id'
explanation	String	Zusätzliche Erläuterungen	Nein	Leere Zeichenkette
type	FieldType	Eingabeart	Ja	Keiner
usage	FieldUsage	Verwendungsart	Nein	INPUT
style	Style	Rasterbasierte Position und Größe	Nein	Leerer Style
hidden	Boolean	Feld ausblenden	Nein	False
highlighted	Boolean	Feld hervorheben	Nein	False

Field (2/2)

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
allowEmpty	Boolean	Leere Eingaben erlauben	Nein	False
datatype	FieldDatatype	Datentyp der Eingaben	Nein	STRING
restrictions	List<Condition>	Weitere Anforderungen an die Eingaben	Nein	Leere Liste
reactions	List<Operation>	Reaktionen bei Änderung der Eingabe	Nein	Leere Liste

Field – Hinweise zu Attributen (1/2)

Attributname	Hinweis
id	• Muss gültiger JavaScript-Variablenname sein und darf nicht mit einem Unterstrich beginnen
usage	• Bei type INFO nicht berücksichtigt, da Infofelder für Benutzer stets unveränderlich sind • Wirkt sich bei type TABLE nur auf die Möglichkeit zum Hinzufügen/Entfernen von Einträgen aus
style	• Kann weggelassen werden, wenn es bei allen Feldern weggelassen wird • Bei inneren Feldern (innerFields) einer Tabelle (type TABLE) nicht berücksichtigt
allowEmpty	• Nur bei usage INPUT berücksichtigt (Eingabebeschränkungen ohne Eingabe sinnlos) • Bei type INFO nicht berücksichtigt, da Infofelder für Benutzer stets unveränderlich sind • Felder mit type CHECK gelten als leer, wenn keine Option ausgewählt ist • Felder mit type SELECT gelten als leer, wenn eine leere Zeichenkette (bei emptyOption FALSE) bzw. die erste Option (bei emptyOption TRUE) gewählt wurde • Felder mit type LIST oder TABLE gelten als leer, wenn alle Einträge leer sind
datatype	• Validierung des Datentyps nur bei usage INPUT (Eingabebeschränkungen ohne Eingabe sinnlos) • Bei type INFO, CHECK, SELECT und TABLE nicht berücksichtigt • Bei type SPLIT und LIST auf jeden Eintrag angewandt • Bei type TEXT, SPLIT und LIST mit datatype NUMBER oder INTEGER: automatische Umwandlung der Werte in Zahlen, leere Zeichenketten werden zu NaN

Field – Hinweise zu Attributen (2/2)

Attributname	Hinweis
restrictions, reactions	• Nur bei usage INPUT berücksichtigt (Eingabebeschränkungen ohne Eingabe sinnlos) • Bei type INFO nicht berücksichtigt, da Infofelder für Benutzer stets unveränderlich sind • Bei type SPLIT, LIST und TABLE kann die Variable "entry" genutzt werden, um die einzelnen Einträge zu adressieren, Bsp.: "entry >= 0" → jeder Eintrag ("entry") muss >= 0 sein • Es werden stets erst die Reaktionen (reactions) durchgeführt, die Validierung, inklusive der Prüfung der Restriktionen (restrictions), erfolgt danach

Field – Hinweis zu inneren Feldern

Innere Felder können nicht direkt referenziert werden

- Beispiel:
  - Globales Feld g1 vom type TEXT
  - Globales Feld g2 vom type TABLE mit inneren Feldern i1 und i2, beide vom type TEXT
- Konsequenzen im Beispiel:
  - g1 und g2 können in ihren Restriktionen auf sich selbst und aufeinander zugreifen
  - g1 und g2 können in ihren Restriktionen nicht direkt auf i1 und i2 zugreifen
  - i1 und i2 können in ihren Restriktionen jeweils nur auf sich selbst direkt zugreifen
  - i1 und i2 können in ihren Restriktionen auf g1 und g2 zugreifen, da diese globale Felder sind
- Lösung:
  - Indirekter Zugriff auf innere Felder über ihr jeweils übergeordnetes Feld
  - Im Beispiel: nutzen der Methoden von g2, um auf i1 und i2 zugreifen zu können

FieldType

Wert	Beschreibung
INFO	Feld ist ein Infofeld (InfoField), Benutzer können keine Eingaben machen, Inhalt wird als Zeichenkette oder Pfad einer Ressource verarbeitet
TEXT	Feld ist ein einfaches Textfeld (TextField), Benutzer geben Inhalt als Text ein, Inhalt wird als Zeichenkette oder Zahl verarbeitet
SPLIT	Feld ist ein aufgeteiltes Textfeld (SplitField), Benutzer geben Inhalt als Text oder Aufzählung ein, Inhalt wird als Liste von Werten (Zeichenketten/Zahlen) verarbeitet
CHECK	Feld ist ein Ankreuzfeld (CheckField), Benutzer wählen zwischen mehreren Ankreuzfeldern, Inhalt wird über die ID oder eine Liste von IDs der gewählten Felder verarbeitet
SELECT	Feld ist ein Auswahlfeld (SelectField), Benutzer wählen eine Option im Auswahlfeld aus, Inhalt wird über die ID oder den Index der gewählten Option verarbeitet
LIST	Feld ist ein Listenfeld (ListField), Benutzer geben Inhalt als Einträge in mehreren Feldern ein, Inhalt wird als Liste von Werten (Zeichenketten/Zahlen) verarbeitet
TABLE	Feld ist ein Tabellenfeld (TableField), Benutzer geben Inhalt als Einträge einer Tabelle ein, Inhalt wird als Liste von Tabelleneinträgen (Objektfelder/ObjectField) verarbeitet

FieldUsage

Wert	Beschreibung
INPUT	Feld dient der Eingabe durch den Benutzer, es ist beschreibbar und Eingaben werden validiert
TEMP	Feld dient der Ausgabe von temporären Werten, es ist daher nicht beschreibbar
OUTPUT	Feld dient der Ausgabe von Endergebnissen, es ist daher nicht beschreibbar

Hinweis: die Unterscheidung von TEMP und OUTPUT ist rein semantisch und wirkt sich im LeMoS nicht aus

FieldDatatype

Wert	Beschreibung
INTEGER	Das Feld soll ausschließlich Ganzzahlen akzeptieren, der Inhalt soll als Zahl ausgelesen werden
NUMBER	Das Feld soll beliebige Zahlen akzeptieren, der Inhalt soll als Zahl ausgelesen werden
STRING	Das Feld soll jede Eingabe akzeptieren, der Inhalt soll als Zeichenkette ausgelesen werden

Style

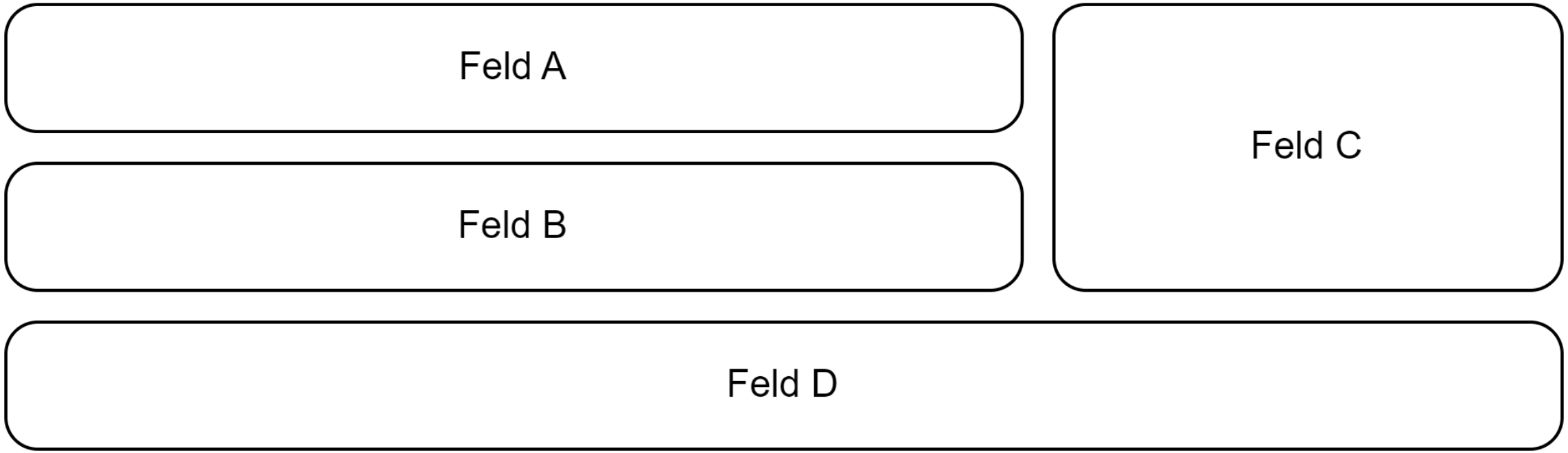
Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
row	Integer	Zeile im Raster, beginnend bei 1	Ja	Keiner
column	Integer	Spalte im Raster, beginnend bei 1	Ja	Keiner
width	Integer	Breite in Rasterfeldern	Nein	1
height	Integer	Höhe in Rasterfeldern	Nein	1

Erläuterung:

- Lernmodulfelder werden an einem gemeinsamen Raster dynamischer Größe ausgerichtet
- Die Position (row + column) entspricht der oberen linken Ecke des Lernmodulfelds
- Die Größe (width + height) entspricht der Anzahl der nach rechts bzw. unten eingenommenen Rasterfelder und sollte entsprechend bei der Position des nächsten Felds berücksichtigt werden
- Beispiel: Lernmodulfeld 1 hat folgende Attribute: row=1, column=1, width=2, height=3
  --> Lernmodulfeld 2 horizontal erst ab row+height=4 und vertikal erst ab oder column+width=3

Style – Beispiel

Bezeichnung	Row	Column	Width	Height
Feld A	1	1	2	1
Feld B	2	1	2	1
Feld C	1	3	1	2
Feld D	3	1	3	1



InfoField (zusätzlich zu Field)

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
text	String	Anzuzeigender Inhalt oder Alternativtext	Nein	Leere Zeichenkette
resource* **	Resource	Anzuzeigende Ressource	Nein	Leere Ressource
resourceID* **	String	ID der anzuzeigenden Ressource	Nein	Leere Zeichenkette
contentType	ContentType	Art des Inhalts	Nein	TEXT
maxWidth*	Integer	Maximale Breite des Inhalts in Pixeln	Nein	-1 (unbegrenzt)
maxHeight*	Integer	Maximale Höhe des Inhalts in Pixeln	Nein	-1 (unbegrenzt)

*: Bei contentType TEXT ignoriert
**: Bei contentType ungleich TEXT ist stets entweder resource oder resourceID zu konfigurieren

CheckField (zusätzlich zu Field)

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
options	List<Option>	Zur Auswahl stehende Optionen	Ja	Keiner
multipleCheck	Boolean	Auswahl mehrerer Optionen erlauben	Nein	true

SelectField (zusätzlich zu Field)

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
options	List<Option>	Zur Auswahl stehende Optionen	Ja	Keiner
emptyOption	Boolean	Erste Option als leer interpretieren	Nein	false

Option

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
id	String	Eindeutige Identifikation, z.B. 'Opt0'	Ja	Keiner
name	String	Anzeigename, z.B. 'Option 1'	Nein	Wert von 'id'
explanation	String	Zusätzliche Erläuterungen	Nein	Leere Zeichenkette

MultilineableField (zusätzlich zu Field)

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
multiline	Boolean	Mehrzeiliges Feld	Nein	False
minLines*	Integer	Minimale Zeilenanzahl	Nein	1
maxLines*	Integer	Maximale Zeilenanzahl	Nein	-1 (unbegrenzt) Minimum: 'minLines'
fixLines*	Integer	Feste Zeilenanzahl	Nein	-1 (unbegrenzt)

*: nur bei multiline = true berücksichtigt

TextField (zusätzlich zu MultilineableField)

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert

SplitField (zusätzlich zu MultilineableField)

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
separator	String	Trennzeichen, an dem die Eingabe aufzuteilen ist	Nein	Leere Zeichenkette (Aufteilung nach jedem Zeichen)

ExpandableField (zusätzlich zu Field)

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
growthDirection	FieldOrientation	Wachstumsrichtung	Nein	VERTICAL
initialEntries	Integer	Initiale Anzahl von Einträgen	Nein	1
maxEntries	Integer	Maximale Anzahl von Einträgen	Nein	-1 (unbegrenzt) Minimum: 'initialEntries'

FieldOrientation

Wert	Beschreibung
HORIZONTAL	Neue Einträge des Felds sollen unter den bisherigen eingefügt werden
VERTICAL	Neue Einträge des Felds sollen rechts neben den bisherigen eingefügt werden

ListField (zusätzlich zu ExpandableField)

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
minEntryWidth*	Integer	Minimale Breite der einzelnen Einträge	Nein	-1 (unbegrenzt)
allowWrap*	Boolean	Zeilenumbruch, falls Felder nicht mehr in Zeile passen	Nein	False

*: nur bei growthDirection HORIZONTAL berücksichtigt

TableField (zusätzlich zu ExpandableField)

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
innerFields	List<Field>	Felder einer Tabellenzeile/-spalte	Ja	Keiner
minColumnWidth*	Integer	Minimale Breite der Tabellenspalten	Nein	-1 (unbegrenzt)
minColumnWidths* **	List<Integer>	Minimale Breiten der Tabellenspalten	Nein	Leere Liste (unbegrenzt)
relativeColumnWidths***	List<Integer>	Relative Breiten der Tabellenspalten	Nein	Leere Liste (alle gleich)

*: Kombinationen von minColumnWidth und minColumnWidths sind nicht zulässig
**: bei growthDirection VERTICAL sind stets so viele Werte wie innere Felder anzugeben,
bei growthDirection HORIZONTAL sind stets 2 Werte (für Kopf und Rest) anzugeben
***: bei growthDirection HORIZONTAL nicht berücksichtigt,
wenn die Inhalte einer Spalte es erforderlich machen, wird sie unabhängig von der Konfiguration dynamisch verbreitert

Modellierungssprache

Objekte für Zustände

State

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
id	String	Eindeutige Identifikation, z.B. 'Z0'	Ja	Keiner
name	String	Anzeigename, z.B. 'N berechnet'	Nein	Wert von 'id'
explanation	String	Erläuterungen zum Zustand	Nein	Leere Zeichenkette (keine Ausgabe)
type	StateType	Art des Zustands	Nein	STATE
importance	StateImportance	Wichtigkeit (kleiner oder große Schritt)	Nein	HIGH
actions	List<Operation>	Vor Zustandseintritt auszuführende Operationen	Nein	Leere Liste
transitions	List<Transition>	Übergänge zu weiteren Zuständen	Nein	Leere Liste
errorMessage	String	Fehlermeldung, falls kein Übergang möglich	Nein	Leere Zeichenkette (keine Fehlermeldung)

State – Hinweise zu Attributen

Attributname	Hinweis
id	• Muss gültiger JavaScript-Variablenname sein und darf nicht mit einem Unterstrich beginnen
name	• Wird ausschließlich in internen Ausgaben verwendet, dient der Fehlersuche
explanation	• Wird über Protokollfeld ausgegeben, dient der Erläuterung für den Benutzer
importance	• Bei type JUNCTION nicht berücksichtigt, stattdessen stets ZERO
actions	• Bei type JUNCTION nicht berücksichtigt, stattdessen stets leere Liste
transitions	• Pro Zustand darf maximal ein Übergang bedingungslos sein
errorMessage	• Um Anweisungen in die Nachricht aufzunehmen, kann die ${}-Syntax von JavaScript verwendet werden • Beispiel einer Konfigurationszeile: *errorMessage: Für ${feld.getValue()} ist keine Ausführung möglich.*

StateType

Wert	Beschreibung
ENTRY	Zustand ist der Einstiegszustand (Initialzustand) des Lernmoduls
STATE	Zustand ist ein klassischer Zustand ohne weitere Besonderheiten
JUNCTION	Zustand ist eine Fallunterscheidung, er führt keine Aktionen durch und wird stets automatisch fortgesetzt (auch bei kleinen Schritten bleibt der Zustandsgraph hier nicht stehen)

Hinweis: das Verhalten des Typs JUNCTION ist durch entsprechende Konfiguration auch mit dem Typ STATE erreichbar, die Verwendung des Typs JUNCTION ist somit rein semantisch zu begründen

StateImportance

Wert	Beschreibung
ZERO	Zustand dient dem internen Ablauf des Zustandsgraphen, Benutzer sollen ihn nicht bemerken
LOW	Zustand stellt einen detaillierten Schritt dar, Benutzer sollen ihn überspringen können*
HIGH	Zustand stellt einen essenziellen Schritt dar, Benutzer sollen ihn nicht überspringen können* **

*: überspringen = nächster Zustandsübergang wird automatisch ausgelöst, die Aktionen des übersprungenen Zustands werden dennoch ausgeführt

**: über die Schaltfläche >| zur Ausführung des Lernmoduls bis zum Ende, kann der Zustand dennoch übersprungen werden

Transition

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
target	String	ID des Zielzustands	Ja	Keiner
name*	String	Anzeigename, z.B. 'T1'	Nein	Leere Zeichenkette
explanation**	String	Erläuterungen zum Übergang	Nein	Leere Zeichenkette (keine Ausgabe)
conditions	List<Condition>	Zu erfüllende Bedingungen	Nein	Leere Liste

*: wird ausschließlich in internen Ausgaben verwendet, dient der Fehlersuche
**: wird über Protokollfeld ausgegeben, dient der Erläuterung für den Benutzer

Hinweis: pro Zustand darf maximal ein Übergang bedingungslos sein

Modellierungssprache

Objekt für Szenarien

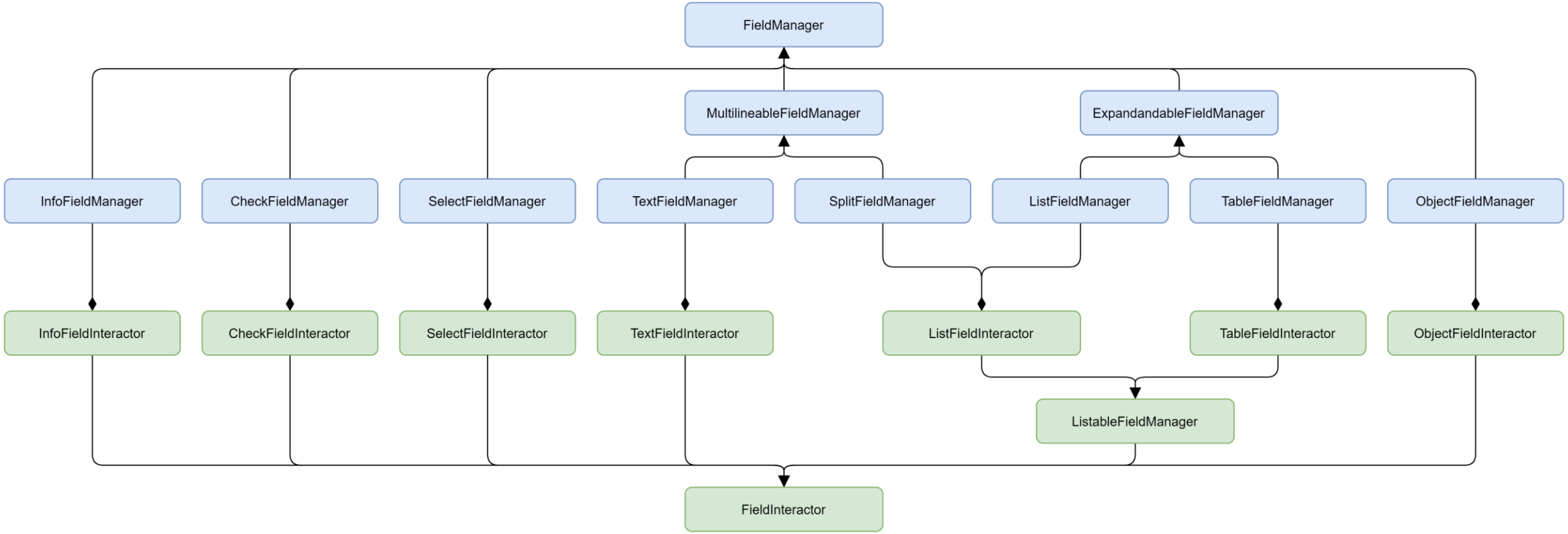
Scenario

Attributname	Datentyp	Beschreibung	Erforderlich	Standardwert
id	String	Eindeutige Identifikation, z.B. 'S1'	Ja	Keiner
name	String	Anzeigename in Szenarioauswahl, z.B. 'Szenario 1'	Nein	Wert von 'id'
explanation	String	Beschreibung des Szenarios, z.B. 'Länge gesucht'	Nein	Leere Zeichenkette
initialization	List<Operation>	Operationen zum Initialisieren des Szenarios	Ja	Keiner

Hinweis: Szenarien sind als eigene Dateien im Unterordner "scenarios" des Lernmoduls abzulegen

– LeMoS – Systemfunktionen

Grundlegende Struktur



Erläuterung: zu jedem Feldtyp gibt es einen Verwalter (Manager, blau) und einen Interaktor (Interactor, grün)
- Die Verwalter sind für interne Aufgaben der Felder zuständig, während die Interaktoren die Systemfunktionen bereitstellen
- Zur Vereinfachung wird an einigen Stellen dieses Dokuments der Anhang "Manager" beziehungsweise "Interaktor" weggelassen
- SplitField und ListField teilen sich einen gemeinsamen Interaktor, weil beide wie eine Liste behandelt werden
- TableFields werden als Liste von Tabelleneinträgen (ObjectField) betrachtet und übernehmen daher viele der Listenfunktionen

Hinweis: Neben den Funktionen der Interaktoren gibt es noch einige statische Systemfunktionen, etwa zur Protokollierung

Systemfunktionen

Feldfunktionen

FieldInteractor

Lesende Methoden

Funktionsname	Parameter	Beschreibung
equals	value	Gibt zurück, ob der Inhalt des Felds mit dem des übergebenen Felds oder dem übergebenen Inhalt übereinstimmt
getDebug	/	Gibt einige Informationen über das Feld zurück, dient zur Fehlersuche
getName	/	Gibt den Namen des Felds zurück
getPrint	/	Gibt den Inhalt des Felds als Ausgabe formatiert zurück
getValue	/	Gibt den Inhalt des Felds zurück Alias: Impliziter Aufruf durch "Feldname.value"
isDisplayed	/	Gibt zurück, ob das Feld sichtbar, das heißt nicht ausgeblendet ist
isEmpty	/	Gibt zurück, ob das Feld leer ist
isEnabled	/	Gibt zurück, ob das Feld beschreibbar, das heißt nicht deaktiviert ist
isHighlighted	/	Gibt zurück, ob das Feld hervorgehoben ist
toString	/	Gibt den Inhalt des Felds als Text zurück Alias für implizite Umwandlung des Felds in Text
valueOf	/	Gibt den Inhalt des Felds ohne weitere Kapselungen zurück Alias für implizite Umwandlung des Felds in einen Wert

FieldInteractor

Verändernde Methoden

Funktionsname	Parameter	Beschreibung
clear	/	Setzt den Inhalt des Felds auf einen leeren Wert
setDisplayed	displayed = true	Setzt die Sichtbarkeit des Felds entsprechend der Übergabe
setEnabled	enabled = true	Setzt die Beschreibbarkeit des Felds entsprechend der Übergabe
setHighlighted	highlighted = true	Setzt die Hervorhebung des Felds entsprechend der Übergabe
setValue	value	Setzt den Inhalt des Felds auf den übergebenen Wert Alias: Impliziter Aufruf durch "Feldname.value = …"

FieldInteractor

Ergänzung zu getValue() und setValue(…)-Methode

Feldtyp	Mögliche Datentypen
InfoField	Anzuzeigender Text oder Pfad zur anzuzeigenden Ressource (abhängig von konfigurierter Eigenschaft contentType)
CheckField	ID der auszuwählenden Option oder Array von IDs der ausgewählten/auszuwählenden Optionen (Rückgabe abhängig von konfigurierter Eigenschaft multipleCheck)
SelectField	ID oder Index der ausgewählten/auszuwählenden Option
TextField	Beliebiger Text oder beliebige Zahl (Rückgabe abhängig von konfigurierter Eigenschaft datatype)
SplitField	Beliebiger Text, beliebige Zahl oder Array von Werten (Rückgabe abhängig von konfigurierter Eigenschaft datatype)
ListField	Array von Werten (Wert = Text oder Zahl, abhängig von konfigurierter Eigenschaft datatype)
TableField	Array von Arrays von Werten, innere Arrays müssen in Länge und Reihenfolge zu den Attributen der Tabelle passen Array von Objekten oder Maps, Objekte/Maps können Attribute in beliebiger Reihenfolge definieren oder weglassen Array von ObjectField-Instanzen (für Rückgabe als **Array von Maps: valueOf()**-Methode verwenden)
ObjectField	Analog zu Inhalten bei TableField: Array, Objekt oder Map

Hinweise:

- alle aufgelisteten Datentypen sind für die setValue(…)-Methode möglich
- die unterstrichenen Datentypen entsprechen den Rückgaben der getValue()-Methode
- die valueOf()-Methode entspricht in der Regel der getValue()-Methode

InfoFieldInteractor (zusätzlich zu FieldInteractor)

Funktionsname	Parameter	Beschreibung

CheckFieldInteractor (zusätzlich zu FieldInteractor)

Funktionsname	Parameter	Beschreibung
getOptions	/	Gibt alle möglichen Optionen des Ankreuzfelds zurück
getCheckedOptions	/	Gibt alle angekreuzten Optionen des Ankreuzfelds zurück
getUncheckedOptions	/	Gibt alle nicht angekreuzten Optionen des Ankreuzfelds zurück
getOptionNames	/	Gibt die Namen aller möglichen Optionen des Ankreuzfelds zurück
getCheckedOptionNames	/	Gibt die Namen aller angekreuzten Optionen des Ankreuzfelds zurück
getUncheckedOptionNames	/	Gibt die Namen aller nicht angekreuzten Optionen des Ankreuzfelds zurück

SelectFieldInteractor (zusätzlich zu FieldInteractor)

Funktionsname	Parameter	Beschreibung
getOptions	/	Gibt alle möglichen Optionen des Auswahlfelds zurück
getOptionNames	/	Gibt die Namen aller möglichen Optionen des Auswahlfelds zurück
getSelectedIndex	/	Gibt den Index der aktuell ausgewählten Option zurück
getSelectedName	/	Gibt den Namen der aktuell ausgewählten Option zurück

TextFieldInteractor (zusätzlich zu FieldInteractor)

Von String (JavaScript) übernommene, lesende Methoden:

Funktionsname	Parameter	Beschreibung
at	index	Gibt Zeichen an der Stelle 'index' zurück
endsWith	searchString, endPosition	Gibt zurück, ob der Feldinhalt auf 'searchString' endet Durch 'endPosition' kann das Ende temporär versetzt werden
includes	searchString, position	Gibt zurück, ob der Feldinhalt 'searchString' beinhaltet*
indexOf	searchString, position	Gibt den Index des ersten Vorkommens von 'searchString' im Feld zurück*
lastIndexOf	searchString, position	Gibt den Index des letzten Vorkommens von 'searchString' im Feld zurück*
match	regexp	Gibt die Treffer von 'regexp' im Feldinhalt zurück**
matchAll	regexp	Gibt die Treffer von 'regexp' im Feldinhalt als Iterator zurück**
search	regexp	Gibt den Index des ersten Treffers von 'regexp' im Feldinhalt zurück**
split	separator, limit	Gibt den durch 'separator' unterteilten Feldinhalt zurück** Durch 'limit' kann die Anzahl der zurückgegebenen Teile begrenzt werden
startsWith	searchString, position	Gibt zurück, ob der Feldinhalt mit 'searchString' anfängt Durch 'position' kann der Start temporär versetzt werden

*: Durch 'position' kann definiert werden, ab welcher Stelle gesucht wird
**: Möglichkeiten für 'regexp' bzw. 'separator': siehe JavaScript-String

TextFieldInteractor (zusätzlich zu FieldInteractor)

Von String (JavaScript) übernommene, verändernde Methoden:

Funktionsname	Parameter	Beschreibung
concat	…str	Hängt 'str' an den bestehenden Feldinhalt an
repeat	count	Setzt den Inhalt auf 'count' Kopien des bisherigen Feldinhalts
replace	pattern, replacement	Ersetzt den ersten Treffer von 'pattern' mit 'replacement'*
replaceAll	pattern, replacement	Ersetzt alle Treffer von 'pattern' mit 'replacement'*
slice	indexStart, indexEnd	Setzt den Inhalt auf den bisherigen Inhalt von 'indexStart' bis 'indexEnd'
toLowerCase	/	Konvertiert den Feldinhalt in ausschließlich Kleinbuchstaben
toUpperCase	/	Konvertiert den Feldinhalt in ausschließlich Großbuchstaben
trim	/	Schneidet die Weißzeichen an Anfang und Ende des Feldinhalts ab
trimEnd	/	Schneidet die Weißzeichen am Ende des Feldinhalts ab
trimStart	/	Schneidet die Weißzeichen am Anfang des Feldinhalts ab

*: Möglichkeiten für pattern und replacement: siehe JavaScript-String

TextFieldInteractor (zusätzlich zu FieldInteractor)

Zusätzlich zu String (JavaScript) implementierte Methoden:

Funktionsname	Parameter	Beschreibung
asNumber	/	Gibt den Feldinhalt als Zahl zurück
getLength	/	Gibt die Länge des Feldinhalts zurück
insert	index, …str	Schiebt 'str' an der Stelle 'index' in den bestehenden Feldinhalt ein
remove	indexStart, indexEnd = indexStart + 1	Entfernt den bisherigen Inhalt von 'indexStart' bis 'indexEnd'
replaceAt	index, ...str	Ersetzt das Zeichen an der Stelle 'index' mit 'str'

ListableFieldInteractor (zusätzlich zu ListableFieldInteractor)

Von Array (JavaScript) übernommene, lesende Methoden:

Funktionsname	Parameter	Beschreibung
at	index	Gibt den Eintrag an der Stelle 'index' zurück
entries	/	Gibt einen Iterator über Index-Eintrag-Paare des Felds zurück
every	callbackFn	Gibt zurück, ob die Funktion 'callbackFn' für alle Einträge wahr ist
find	callbackFn	Gibt den ersten Eintrag zurück, für den die Funktion 'callbackFn' wahr ist
findIndex	callbackFn	Gibt den ersten Index zurück, an dem die Funktion 'callbackFn' wahr ist
findLast	callbackFn	Gibt den letzten Eintrag zurück, für den die Funktion 'callbackFn' wahr ist
findLastIndex	callbackFn	Gibt den letzten Index zurück, an dem die Funktion 'callbackFn' wahr ist
forEach	callbackFn	Führt die Funktion 'callbackFn' für alle Einträge des Felds aus
join	separator	Gibt die Einträge des Felds durch 'seperator' getrennt als Text zurück
keys	/	Gibt einen Iterator über die Indizes der Einträge des Felds zurück
some	callbackFn	Gibt zurück, ob die Funktion 'callbackFn' für min. einen Eintrag wahr ist
values	/	Gibt einen Iterator über die Einträge des Felds zurück

ListableFieldInteractor (zusätzlich zu ListableFieldInteractor)

Von Array (JavaScript) übernommene, verändernde Methoden:

Funktionsname	Parameter	Beschreibung
concat	…value	Hängt 'value' an die bestehenden Einträge an
fill	value, start, end	Setzt den Wert aller Einträge von 'start' bis 'end' auf 'value'
filter	callbackFn	Entfernt alle Einträge, die die Funktion 'callbackFn' nicht erfüllen
pop	/	Entfernt den letzten Eintrag des Felds
map	callbackFn	Ersetzt alle Einträge durch ihr jeweiliges Ergebnis der Funktion 'callbackFn'
push	…element	Hängt 'element' an die bestehenden Einträge an
reverse	/	Kehrt die Reihenfolge der Einträge um
shift	/	Entfernt den ersten Eintrag des Felds
slice	start, end	Setzt den Inhalt auf den bisherigen Inhalt von 'start' bis 'end'
sort	compareFn	Sortiert die Einträge entsprechend der Vergleichsfunktion 'callbackFn'
splice	start, deleteCount, …item	Entfernt 'deleteCount' Einträge ab 'start' und/oder fügt ab 'start' die neuen Einträge 'item' ein
unshift	…element	Schiebt 'element' als erste Einträge ein

ListableFieldInteractor (zusätzlich zu ListableFieldInteractor)

Von Array (JavaScript) übernommene, angepasste Methoden:

Funktionsname	Parameter	Beschreibung
includes	searchElement, fromIndex	Gibt zurück, ob mindestens ein Eintrag dem 'searchElement' entspricht*
indexOf	searchElement, fromIndex	Gibt den Index des ersten Eintrags, der 'searchElement' entspricht, zurück*
lastIndexOf	searchElement, fromIndex	Gibt den Index des letzten Eintrags, der 'searchElement' entspricht, zurück*

*: Durch 'fromIndex' kann definiert werden, ab welcher Stelle gesucht wird

Hinweis: der Unterschied zwischen den hier aufgelisteten Methoden und denen der Array-Klasse von JavaScript liegt im Vergleich der Einträge, der im LeMoS durch eine equals-Methode ersetzt wurde (normalerweise via "===")

ListableFieldInteractor (zusätzlich zu ListableFieldInteractor)

Zusätzlich zu Array (JavaScript) implementierte Methoden:

Funktionsname	Parameter	Beschreibung
add	…element	Alias zur Methode 'push'
contains	searchElement, fromIndex	Alias zur Methode 'includes'
first	/	Gibt den ersten Eintrag des Felds zurück
getLength	/	Gibt die Anzahl der Einträge des Felds zurück
insert	start, ...item	Fügt ab 'start' die neuen Einträge 'item' ein
last	/	Gibt den letzten Eintrag des Felds zurück
remove	searchElement	Entfernt den ersten Eintrag, der 'searchElement' entspricht
removeIndex	start, end = start + 1	Entfernt alle Einträge von start bis end
set	index, ...item	Ersetzt den Eintrag an der Stelle 'index' mit den Einträgen 'item'
unify	/	Entfernt alle doppelten Einträge des Felds
withEmptyEntries	/	Gibt einen Interaktor desselben Felds zurück, in dem die leeren Einträge berücksichtigt werden

ListableFieldInteractor (zusätzlich zu ListableFieldInteractor)

Zusätzlich zu Array (JavaScript) implementierte, iterierende Methoden:

Funktionsname	Parameter	Beschreibung
current	/	Gibt den Eintrag an der aktuellen Position zurück
currentIndex	/	Gibt den Index der aktuellen Position zurück
findNext	callbackFn	Gibt den nächsten Eintrag zurück, der die Funktion 'callbackFn' erfüllt
findPrevious	callbackFn	Gibt den vorherigen Eintrag zurück, der die Funktion 'callbackFn' erfüllt
getIteratorPosition	/	Gibt die aktuelle Position des Iterators zurück
hasNext	/	Gibt zurück, ob es einen nächsten Eintrag gibt
hasPrevious	/	Gibt zurück, ob es einen vorherigen Eintrag gibt
isAtEnd	/	Gibt zurück, ob die aktuelle Position dem Ende des Felds entspricht
isAtStart	/	Gibt zurück, ob die aktuelle Position dem Anfang des Felds entspricht

ListableFieldInteractor (zusätzlich zu ListableFieldInteractor)

Zusätzlich zu Array (JavaScript) implementierte, iterierende Methoden:

Funktionsname	Parameter	Beschreibung
next	/	Gibt den nächsten Eintrag zurück
previous	/	Gibt den vorherigen Eintrag zurück
restartIterator	/	Startet die Iteration von vorne
setIteratorPosition	position	Setzt die aktuelle Position des Iterators auf 'position'
setToEnd	/	Setzt die aktuelle Position auf das Ende des Felds
setToStart	/	Setzt die aktuelle Position auf den Anfang des Felds

ListFieldInteractor (zusätzlich zu ListableFieldInteractor)

Funktionsname	Parameter	Beschreibung
average	/	Gibt den Durchschnitt aller Einträge des Listenfelds zurück*
maximum	/	Gibt das Maximum aller Einträge des Listenfelds zurück*
median	/	Gibt den Median aller Einträge des Listenfelds zurück*
minimum	/	Gibt das Minimum aller Einträge des Listenfelds zurück*
sum	/	Gibt die Summe aller Einträge des Listenfelds zurück*

*: Einträge müssen zu einer Zahl konvertierbar sein

TableFieldInteractor (zusätzlich zu ListableFieldInteractor)

Funktionsname	Parameter	Beschreibung
fillAttribute	attribute, value, start = 0, end = length()	Setzt für alle Einträge von 'start' bis 'end' den Wert des Attributs 'attribute' auf 'value'
findByAttribute	attribute, value	Gibt den ersten Eintrag zurück, dessen Attribut 'attribute' den Wert 'value' hat
findIndexByAttribute	attribute, value	Gibt den ersten Index zurück, an dem das Attribut 'attribute' den Wert 'value' hat
findLastByAttribute	attribute, value	Gibt den letzten Eintrag zurück, dessen Attribut 'attribute' den Wert 'value' hat
findLastIndexByAttribute	attribute, value	Gibt den letzten Index zurück, an dem das Attribut 'attribute' den Wert 'value' hat
findNextByAttribute	attribute, value	Gibt den nächsten Eintrag zurück, dessen Attribut 'attribute' den Wert 'value' hat
findPreviousByAttribute	attribute, value	Gibt den vorherigen Eintrag zurück, dessen Attribut 'attribute' den Wert 'value' hat
getAttribute	attribute	Gibt die Werte des Attributs 'attribute' aller Einträge zurück
setAttribute	attribute, values	Setzt die Werte des Attributs 'attribute' aller Einträge auf die Werte von 'values' (Länge von 'values' muss der Anzahl der Einträge entsprechen)

ObjectFieldInteractor (zusätzlich zu FieldInteractor)

Von Map (JavaScript) übernommene, lesende Methoden:

Funktionsname	Parameter	Beschreibung
entries	/	Gibt einen Iterator über Name-Feld-Paare des Tabelleneintrags zurück*
forEach	callbackFn	Führt die Funktion 'callbackFn' für Felder des Tabelleneintrags aus
get	key	Gibt einen Interaktor für das Feld mit dem Namen 'key' zurück*
has	key	Gibt zurück, ob der Tabelleneintrag ein Feld mit dem Namen 'key' hat*
keys	/	Gibt einen Iterator über die Namen der Felder des Tabelleneintrags zurück*
values	/	Gibt einen Iterator über die Felder des Tabelleneintrags zurück

*: Der Name eines Felds entspricht der konfigurierten ID des Felds

Hinweis: ObjectFields referenzieren stets einen festen Index der zugehörigen Tabelle, daraus ergibt sich folgendes:

- Das Hinzufügen oder Entfernen von Tabelleneinträgen kann den Inhalt von ObjectField-Instanzen verändern
- Eine ObjectField-Instanz, die den letzten Eintrag einer Tabelle referenziert, wird ungültig, wenn ein Eintrag entfernt wird

ObjectFieldInteractor (zusätzlich zu FieldInteractor)

Von Map (JavaScript) übernommene, verändernde Methoden:

Funktionsname	Parameter	Beschreibung
delete	key	Entfernt den Inhalt des Felds mit dem Namen 'key'*
set	key, value	Setzt den Inhalt des Felds mit dem Namen 'key' auf 'value'*

*: Der Name eines Felds entspricht der konfigurierten ID des Felds

Hinweis: ObjectFields referenzieren stets einen festen Index der zugehörigen Tabelle, daraus ergibt sich folgendes:

- Das Hinzufügen oder Entfernen von Tabelleneinträgen kann den Inhalt von ObjectField-Instanzen verändern
- Eine ObjectField-Instanz, die den letzten Eintrag einer Tabelle referenziert, wird ungültig, wenn ein Eintrag entfernt wird

ObjectFieldInteractor (zusätzlich zu FieldInteractor)

Zusätzlich zu Map (JavaScript) implementierte Methoden:

Funktionsname	Parameter	Beschreibung
getLength	/	Gibt die Anzahl der Felder des Tabelleneintrags zurück
remove	key	Alias zur Methode 'delete'

*: Der Name eines Felds entspricht der konfigurierten ID des Felds

Hinweis: ObjectFields referenzieren stets einen festen Index der zugehörigen Tabelle, daraus ergibt sich folgendes:

- Das Hinzufügen oder Entfernen von Tabelleneinträgen kann den Inhalt von ObjectField-Instanzen verändern
- Eine ObjectField-Instanz, die den letzten Eintrag einer Tabelle referenziert, wird ungültig, wenn ein Eintrag entfernt wird

GlobalVariable

Funktionsname	Parameter	Beschreibung
getName	/	Gibt den Namen der Variable zurück
getValue	/	Gibt den Inhalt der Variable zurück Alias: Impliziter Aufruf durch "Variablenname.value"
toString	/	Gibt den Inhalt der Variable als Text zurück Alias für implizite Umwandlung der Variable in Text
setValue	value	Setzt den Inhalt des Felds auf den übergebenen Wert Alias: Impliziter Aufruf durch "Variablenname.value = …"
valueOf	/	Gibt den Inhalt der Variable ohne weitere Kapselungen zurück Alias für implizite Umwandlung der Variable in einen Wert

Systemfunktionen

Statische Funktionen

Module

Funktionsname	Parameter	Beschreibung
setLogger	logger	Definiert, welches Feld für die Protokollausgabe verwendet wird Kann genutzt werden, um das Protokoll frei im Lernmodul zu positionieren Hinweis: das Feld sollte ein mehrzeiliges SplitField mit '\n' als Separator sein
clearLogger	/	Setzt den Inhalt des Felds auf einen leeren Wert
clearNoteField	/	Entfernt alle Hinweismeldungen (von alert, fail und error)
log	content	Protokolliert ausschließlich in der Konsole, für interne Ausgaben gedacht
print	content	Protokolliert in Konsole und Protokollfeld, für allgemeine Infos gedacht
alert	content	Protokolliert in der Konsole und als Hinweismeldung, für wichtige Hinweise gedacht
fail	content	Erzeugt eine "schwache" Fehlermeldung, für Eingabefehler gedacht*
error	content	Erzeugt eine "starke" Fehlermeldung, für interne Fehler gedacht*

*: Der Parameter 'content' kann ein Fehlertext oder eine Error-Instanz sein, bei Fehlertexten wird eine neue Error-Instanz erzeugt, bei übergebenen Error-Instanzen wird diese zur Fehlerbehandlung weitergeworfen

Hinweis: alle Fehlermeldungen werden automatisch in der Konsole und als Hinweismeldung ausgegeben, das gilt insbesondere auch für Fehler, die nicht über die Funktionen der Module-Klasse erzeugt wurden

Fields

Funktionsname	Parameter	Beschreibung
countFilled	/	Gibt die Anzahl der befüllten (nicht leeren) Eingabefelder zurück
areFilled	...fields	Gibt zurück, ob alle übergebenen Felder befüllt (nicht leer) sind

States

Funktionsname	Parameter	Beschreibung
getStepCount	/	Gibt die Anzahl der bisherigen Schritte zurück
getCurrentStateID	/	Gibt die ID des aktuellen Zustands zurück
getCurrentStateName	/	Gibt den Namen des aktuellen Zustands zurück
getCurrentStateExplanation	/	Gibt die Erklärungen zum aktuellen Zustand zurück
getLastStateID*	/	Gibt die ID des vorherigen Zustands zurück
getLastStateName*	/	Gibt den Namen des vorherigen Zustands zurück
getLastStateExplanation*	/	Gibt die Erklärungen zum vorherigen Zustand zurück
getLastTransitionName*	/	Gibt den Namen des letzten Übergangs zurück
getLastTransitionExplanation*	/	Gibt die Erklärungen zum letzten Übergang zurück

*: gibt undefined zurück, wenn kein vorheriger Zustand / letzter Übergang existiert (Initialzustand)

Resource

Funktionsname	Parameter	Beschreibung
get	id	Gibt die unter der übergebenen ID konfigurierte Ressource zurück

Utils

Funktionsname	Parameter	Beschreibung
equals	object1, object2	Gibt zurück, ob die Inhalte der übergebenen Objekte gleich sind Entspricht der equals-Methode von FieldInteractor Dient zum universellen Vergleich zweier Objekte, wobei auch beide Objekte JavaScript-Objekten sein können
objectToPrint	object	Gibt den Inhalt von 'object' als Ausgabe formatiert zurück Entspricht der getPrint-Methode von FieldInteractor beziehungsweise den toString-Methoden von Array und Map, da diese angepasst wurden Dient zur universellen Umwandlung verschiedener Objekte in Ausgaben, insbesondere auch für direkte Instanzen von Object